



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Desarrollo De Una Herramienta De Visualización De Alto Rendimiento Para Datos De Radar Meteorológico En Formato ARCO/Zarr, Empleando Mallas No Estructuradas

Trabajo de investigación para optar el título de
Magíster en Ciencias de la Información y las Comunicaciones

Danny Fabián Mora Peña

Director: Cesar Andrey Perdomo Charry, PhD

Universidad Distrital Francisco José De Caldas
Maestría en Ciencias de la Información y las Comunicaciones
Énfasis en Sistemas de Información
Bogotá D. C. – Colombia
2026

Índice general

1. Introducción	4
1.1. Planteamiento del Problema	4
1.2. Hipótesis	5
1.3. Objetivos	5
1.4. Motivación y Contexto Institucional	6
1.5. Organización del Documento	6
2. Marco de Referencia	8
2.1. Marco Teórico	8
2.1.1. El radar meteorológico Doppler y su geometría de barrido	8
2.1.2. Herramientas de código abierto para radar: estado del arte y limitaciones	9
2.1.3. Formatos de datos ARCO y la herramienta raw2zarr	9
2.1.4. WebAssembly como plataforma de cómputo de alto rendimiento en el navegador	10
2.1.5. Mallas no estructuradas en visualización científica	10
2.2. Marco Conceptual	11
2.3. Marco Espacial	12
2.4. Marco Temporal	12
3. Arquitectura de Datos	13
3.1. Análisis de la Estructura del <i>Store</i> Icechunk/Zarr	13
3.2. El Problema de la Incompatibilidad de Icechunk con JavaScript	14
3.3. Decisión de Diseño: Servidor HTTP como Puente	14
3.4. Soporte Multi-Radar: Arquitectura Genérica	14
3.5. API del Servidor	16
3.6. Rendimiento del Pipeline de Datos	18
3.7. Flujo Completo: del <i>Store</i> Zarr a la GPU	18
4. Mallas No Estructuradas	21
4.1. Motivación: La Geometría Polar del Radar	21
4.2. Las Tres Estrategias de Malla Implementadas	21
4.2.1. Representación A: Quads Instanciados (<i>Quad Instanced</i>)	21
4.2.2. Representación B: Triangle Fan (Vértices Explícitos)	25

4.2.3.	Representación C: Triangulación de Delaunay	27
4.3.	Comparación de las Tres Representaciones	28
4.3.1.	Análisis y Elección de Representación para Tiempo Real	29
5.	Renderizado Interactivo	31
5.1.	Arquitectura de Procesamiento Paralelo: Web Workers	31
5.1.1.	Protocolo de Mensajes	31
5.1.2.	Transferable Objects y Eficiencia de Memoria	32
5.1.3.	Resolución de la Condición de Carrera en el Arranque	32
5.2.	Módulo WASM: Compilación Rust → WebAssembly	33
5.3.	Pipeline WebGL 2: del Buffer a los Píxeles	34
5.3.1.	Shader de vértices: Quad instanciado	34
5.3.2.	Shader de vértices: Triangle Fan y Delaunay	34
5.3.3.	Shader de fragmentos: Colormap NWS	35
5.3.4.	Integración con MapLibre GL JS	35
5.4.	Rendimiento del Pipeline de Renderizado	35
6.	Interfaz de Usuario	37
6.1.	Principios de Diseño	37
6.2.	Componentes de la Interfaz	37
6.2.1.	Mapa Base y Capa Radar	37
6.2.2.	Panel de Control Superior	39
6.2.3.	Panel de Información	40
6.2.4.	Leyenda de Colores	40
6.3.	Selector de Sweeps y Elevaciones	40
6.4.	Animación Temporal	41
6.5.	Archivos Locales	41
7.	Evaluación y Validación	42
7.1.	Diseño Experimental	42
7.1.1.	Hardware de Referencia	42
7.1.2.	Conjunto de Datos	42
7.1.3.	Metodología de Medición	43
7.2.	Benchmark de Py-ART	43
7.3.	Benchmark de radar-viz	44
7.3.1.	Pipeline completo: primer cuadro (sin caché)	44
7.3.2.	Benchmark de Mallas (Recompute desde Caché)	44

7.4. Comparación Global: radar-viz vs. Py-ART	45
7.4.1. Discusión	45
7.4.2. Limitaciones del Experimento	45
8. Conclusiones	47
9. Principales Aportes	50
10. Recomendaciones y Futuros Trabajos	52
Referencias	54

1. Introducción

El monitoreo meteorológico en tiempo real constituye una necesidad operativa crítica para los sistemas de alertas tempranas, la gestión del riesgo y la toma de decisiones en contextos donde los fenómenos hidrometeorológicos extremos tienen consecuencias directas sobre la población. En Colombia, la compleja topografía andina y la interacción de los sistemas de circulación oceánica y atmosférica generan patrones de precipitación altamente heterogéneos (Montealegre Bocanegra & Pabón Caicedo, 2000), que exigen herramientas de análisis con alta resolución espacio-temporal.

El radar meteorológico Doppler es la principal fuente de información para detectar y caracterizar fenómenos de precipitación con cobertura espacial amplia y resolución temporal alta. Cada volumen de escaneo (VCP, *Volume Coverage Pattern*) produce centenares de megabytes de datos estructurados en coordenadas esféricas: para el radar KVNK (Vance AFB, Oklahoma) bajo el VCP-12, un solo barrido de elevación (*sweep*) en la elevación mínima de $0,48^\circ$ contiene 720 azimuths $\times$ $1,832$ *range gates*, lo que resulta en 1,319,040 mediciones individuales por variable, por tiempo, por nivel de elevación (Doviak & Zrnić, 2006). Para el radar Guaviare del IDEAM (San José del Guaviare, Colombia), operado en formato IRIS/Sigmet, un barrido equivalente contiene 360 azimuths $\times$ 747 *range gates* con resolución radial de 300 m, lo que permite comparar directamente la herramienta sobre datos de redes de radar heterogéneas.

A pesar de la abundancia de datos, la visualización interactiva de estos volúmenes continúa siendo un cuello de botella. Las herramientas de código abierto ampliamente adoptadas (Py-ART (Helmus & Collis, 2016) y wradlib (Heistermann et al., 2015)) concentran sus fortalezas en la lectura, el preprocesamiento y el análisis cuantitativo, pero su modelo de renderizado basado en Matplotlib y funciones de trazado vectorial incurre en latencias que impiden la exploración interactiva: la lectura de un archivo NEXRAD Level II completo requiere del orden de 1 300 ms y la generación de un PPI estático toma entre 100 y 150 ms adicionales (mediciones propias sobre el conjunto de datos KVNK, detalladas en el Capítulo 7). Esta latencia acumulada rompe el flujo de análisis interactivo. Para mitigar esto, es una práctica común en la investigación pre-renderizar secuencias de imágenes estáticas de forma asíncrona para luego componerlas en animaciones (e.g., archivos GIF o MP4) que simulan continuidad temporal. En contraste, nuestra implementación elimina la necesidad de este costoso flujo de trabajo fuera de línea, habilitando una exploración visual verdaderamente interactiva, fluida y en tiempo real sobre el volumen de datos.

1.1 Planteamiento del Problema

El problema central de esta investigación se sitúa en la intersección de dos desarrollos recientes:

Primero, la aparición de infraestructuras de datos ARCO (*Analysis-Ready, Cloud-Optimized*) para radar meteorológico. La herramienta `raw2zarr` (Ladino, 2024) transforma archivos crudos de radar (formatos NEXRAD, IRIS/Sigmet, ODIM HDF5) en almacenes Zarr v3 gestionados por Icechunk (Earthmover, 2024), habilitando lectura paralela directa desde la nube y eliminando gran parte de la latencia de entrada/salida que caracteriza al flujo de trabajo tradicional con archivos crudos.

Segundo, la madurez de las tecnologías de visualización científica en el navegador web. WebAssembly (World Wide Web Consortium (W3C), 2019) permite ejecutar código de alto rendimiento compilado desde Rust directamente en el *browser*, alcanzando velocidades de ejecución cercanas a las de las aplicaciones nativas de escritorio, superando así las limitaciones históricas de rendimiento de los entornos basados exclusivamente en JavaScript. Por su parte, la API WebGL 2 (Khronos Group, 2017) expone capacidades avanzadas de renderizado por GPU, incluyendo renderizado instanciado (*instanced rendering*) y el uso de búferes de elementos (*Element Buffer Objects*), características de hardware gráfico que no estaban disponibles para aplicaciones web hace menos de una década.

Sin embargo, no existen soluciones consolidadas que integren ambos avances: un sistema que lea datos de radar desde un *store* ARCO/Zarr, construya una representación geométrica apropiada para la naturaleza polar de los datos, y los renderice de forma interactiva a más de 15 FPS en el navegador, sin requerir instalación de software específico ni acceso a supercomputadoras.

1.2 Hipótesis

Representar los barridos de radar meteorológico mediante mallas no estructuradas y renderizarlos a partir de datos en formato Zarr/ARCO, utilizando técnicas de computación acelerada por GPU vía WebAssembly, permite reducir el tiempo total de visualización interactiva y alcanzar una velocidad de refresco superior a 15 FPS en comparación con las herramientas basadas en grillas estructuradas (Py-ART, wradlib), sin incrementar el error medio de interpolación espacial.

1.3 Objetivos

Objetivo General Desarrollar una herramienta interactiva para la visualización eficiente de datos de radar meteorológico en formato Zarr, empleando un esquema de renderizado basado en mallas no estructuradas, con el fin de lograr mayor eficiencia y fidelidad en el análisis de fenómenos atmosféricos.

Objetivos Específicos

1. Adaptar los flujos de datos de radares para su uso eficiente en entornos de visualización,

integrando el pipeline de lectura de *stores* ARCO/Zarr.

2. Investigar y seleccionar algoritmos de generación de mallas no estructuradas apropiados para la geometría polar de los barridos PPI.
3. Desarrollar componentes de renderizado interactivo sobre mallas no estructuradas, con aceleración por GPU vía WebAssembly y WebGL.
4. Desarrollar una interfaz de usuario interactiva para la exploración de datos meteorológicos, que soporte navegación fluida, selector de variable y animación temporal.
5. Evaluar la precisión y eficiencia de la herramienta propuesta en comparación cuantitativa con las soluciones existentes.

1.4 Motivación y Contexto Institucional

Esta investigación se desarrolla en el marco del grupo LASER (Laboratorio en Automatización, Sistemas Embebidos y Robótica) de la Universidad Distrital Francisco José de Caldas, dentro de la línea de Análisis de Datos Meteorológicos. La motivación práctica se origina en las necesidades del Instituto de Hidrología, Meteorología y Estudios Ambientales (IDEAM), que opera una red de radares meteorológicos en Colombia con cobertura sobre zonas de alto riesgo hidrológico. Mejorar la capacidad de visualización de estos datos tiene impacto directo en la calidad de las alertas tempranas y en la seguridad de las comunidades vulnerables. En este trabajo, la herramienta desarrollada se validó sobre el radar Guaviare del IDEAM (un sistema IRIS/Sigmet con estructura de *store* distinta a NEXRAD) demostrando que el pipeline es genérico y aplicable a la red de radares colombiana sin modificaciones al motor de renderizado (véase la Figura 3.1).

Desde una perspectiva académica, esta investigación se alinea con la filosofía del ecosistema Pangeo (Stern et al., 2021), orientada a impulsar una ciencia geoespacial abierta, reproducible y escalable. En congruencia con estos principios, todos los recursos derivados de este proyecto (código fuente, conjuntos de datos de prueba y resultados de las evaluaciones de rendimiento) serán publicados íntegramente bajo una licencia de código abierto.

1.5 Organización del Documento

El presente documento se organiza en nueve capítulos. El **Capítulo 2** presenta el marco de referencia teórico y conceptual. El **Capítulo 3** describe la arquitectura de datos adoptada para la integración con *stores* ARCO/Zarr. El **Capítulo 4** desarrolla las tres estrategias de malla no estructurada diseñadas e implementadas. El **Capítulo 5** detalla los componentes de renderizado interactivo basados en WebAssembly y WebGL. El **Capítulo 6** describe la interfaz de usuario.

El **Capítulo 7** presenta la evaluación cuantitativa y la comparación con Py-ART. Finalmente, los capítulos de **Conclusiones**, **Aportes** y **Recomendaciones** sintetizan los hallazgos y proyectan líneas de trabajo futuro.

2. Marco de Referencia

2.1 Marco Teórico

2.1.1 El radar meteorológico Doppler y su geometría de barrido El radar meteorológico Doppler es un sistema de teledetección activa que emite pulsos electromagnéticos en la banda C (5,6 cm) o banda S (10 cm) y mide la energía retrodispersada por las hidrometeoros para estimar reflectividad Z , velocidad radial V_r y parámetros de polarimetría (ZDR , ρ_{HV} , K_{DP} , Φ_{DP}) (Doviak & Zrnić, 2006).

Un volumen de escaneo completo (*Volume Coverage Pattern*, VCP) consiste en una secuencia de barridos circulares (*sweeps* o cortes de elevación), cada uno realizado a un ángulo de elevación fijo. Aunque la herramienta de visualización desarrollada es agnóstica a la estrategia de escaneo y es capaz de procesar cualquier patrón volumétrico, esta investigación se emplearon diferentes VCPs (VCP-11/12/21/31/32/35/112/212/213) como caso de estudio. En este patrón específico, el radar realiza 17 elevaciones consecutivas desde 0.48° hasta 19.51° . En cada elevación, la antena completa una rotación de 360° tomando mediciones a intervalos regulares de azimut (0.5° en modo de súper resolución), conformando 1 832 celdas de resolución por rayo con una separación radial de 250 m entre celdas. La Figura 2.1 ilustra la geometría resultante.

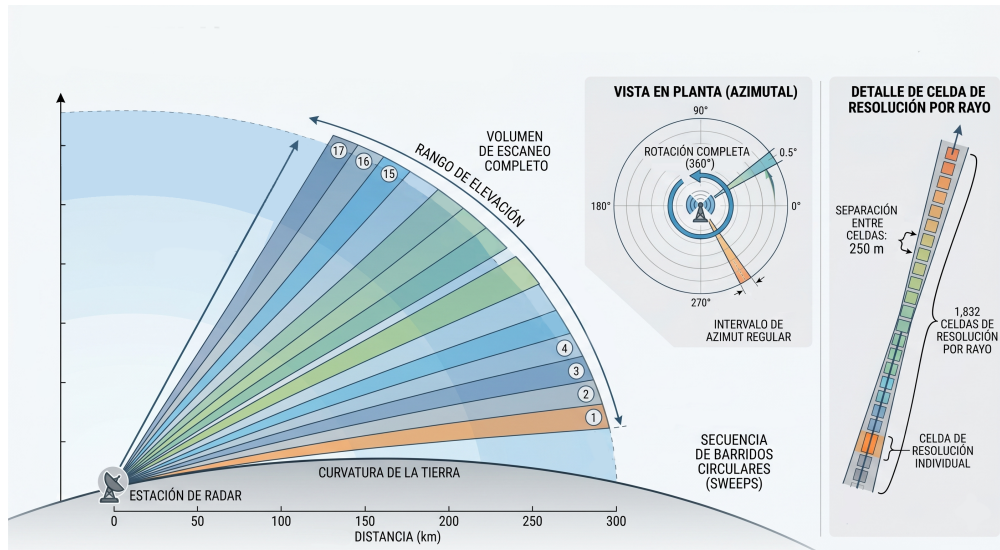


Figura 2.1: Geometría del escaneo radar. **Izquierda:** vista en elevación de los 17 barridos del VCP, **Centro:** vista en planta (azimutal) de un barrido PPI completo; la antena realiza una rotación de 360° con un intervalo regular de $\delta\phi = 0,5^\circ$ entre rayos consecutivos. **Derecha:** detalle de un rayo individual, compuesto por 1,832 celdas de resolución (gates) con una separación radial constante de $\delta r = 250$ m.

La estructura cónica de los barridos es la característica geométrica que motiva el uso de mallas no estructuradas en esta tesis: los gates cercanos al radar tienen dimensiones del orden de $250 \text{ m} \times 0,5^\circ$ ($\approx 218 \text{ m}$ a 25 km de rango), mientras que los gates distantes alcanzan $250 \text{ m} \times 0,5^\circ$ ($\approx 2,100 \text{ m}$ a 240 km). Forzar estos datos a una grilla cartesiana uniforme requiere interpolación o introduce celdas vacías en la periferia; una malla que respete la geometría polar original evita ambos problemas (Kristof, 2010).

2.1.2 Herramientas de código abierto para radar: estado del arte y limitaciones La comunidad meteorológica dispone de un ecosistema de herramientas de análisis de radar en Python, entre las que destacan Py-ART (Helmus & Collis, 2016) y wradlib (Heistermann et al., 2015). Ambas implementan lectores para múltiples formatos de archivo (NEXRAD Level II, ODIM HDF5, IRIS/Sigmat, etc.), rutinas de corrección de datos (atenuación, clutter, desdoble de fase) y funciones de visualización basadas en Matplotlib.

Sin embargo, el modelo de renderizado de estas herramientas presenta limitaciones estructurales:

- **Renderizado vectorial sobre CPU:** Matplotlib (mediante su motor de renderizado estándar Agg) convierte la geometría polar en polígonos vectoriales que la CPU rasteriza de forma secuencial. Para 1,319,040 celdas, este proceso genera millones de operaciones de cálculo ligadas a la CPU (*CPU-bound*), limitando el rendimiento al no aprovechar las capacidades de paralelismo de la GPU (Hunter, 2007).
- **Ausencia de interactividad fluida:** El modelo *plot-then-show* de Matplotlib produce imágenes estáticas; cada modificación (zoom, cambio de variable, avance temporal) requiere regenerar la figura completa.
- **Alta latencia de carga:** La lectura de un archivo NEXRAD Level II completo con Py-ART 2.2.0 sobre el conjunto de datos KVNx tomó $\bar{x} = 1,326 \text{ ms}$ con memoria pico de $\approx 450 \text{ MB}$, según las mediciones realizadas en esta investigación (Capítulo 7).

Comparaciones publicadas por Franda et al. (2023) reportan tiempos de trazado de polígonos vectoriales superiores a 50 s para conjuntos de datos de resolución alta usando HoloViz, y destacan que la rasterización GPU es significativamente más rápida y menos sensible al tamaño de la grilla.

2.1.3 Formatos de datos ARCO y la herramienta raw2zarr Los formatos ARCO (*Analysis-Ready, Cloud-Optimized*) organizan los datos geoespaciales en estructuras optimizadas para el acceso distribuido en la nube, eliminando la necesidad de descargar archivos completos antes de procesarlos (Parente, 2021; Stern et al., 2021).

El formato **Zarr** (Zarr Developers, 2024) es el estándar para arrays n-dimensionales: organiza los datos en *chunks* comprimidos con índices que permiten solicitar solo las regiones necesarias y procesarlas en paralelo. En el contexto de radar, un store Zarr para el conjunto KVNx con 54 timesteps organiza cada variable tridimensional (tiempo $\times$ azimuth $\times$ rango) en chunks que pueden leerse independientemente desde la nube o el disco local.

La herramienta **raw2zarr** (Ladino, 2024) automatiza la conversión de datos crudos de radar al formato Zarr, incorporando además el sistema de versionado **Icechunk** (Earthmover, 2024), una capa de control de versiones al estilo Git sobre *stores* Zarr v3. La arquitectura interna de **raw2zarr** produce un **DataTree** jerárquico ($VCP \rightarrow sweep_N \rightarrow variable$) que preserva la estructura de barrido del radar y constituye el punto de entrada de la presente investigación.

2.1.4 WebAssembly como plataforma de cómputo de alto rendimiento en el navegador WebAssembly (WASM) (World Wide Web Consortium (W3C), 2019) es un formato de instrucciones binario de bajo nivel, diseñado como objetivo de compilación para lenguajes como C, C++ y Rust, que se ejecuta en el navegador a velocidades cercanas a las del código nativo. A diferencia de JavaScript, WASM es tipado estáticamente y se distribuye en un formato binario pre-optimizado que permite al navegador realizar una compilación en flujo (*streaming compilation*) casi instantánea, eliminando así la sobrecarga computacional (*overhead*) de la interpretación de texto y de los costosos ciclos de optimización especulativa del compilador JIT.

Para esta investigación, WASM se usa como plataforma de ejecución de la lógica de transformación geométrica (coordenadas polares $\rightarrow$ Web Mercator $\rightarrow$ vértices GPU). El módulo compilado desde Rust pesa 31 KB en la configuración `--release` con todas las optimizaciones activadas, y su función principal `sweep_to_vertices()` procesa 1,319,040 gates en aproximadamente 13 ms (medición propia sobre Apple M4 Pro, Chrome 125).

2.1.5 Mallas no estructuradas en visualización científica Una **malla no estructurada** es un conjunto de nodos cuyos vecinos se definen mediante una lista explícita de conectividades, en contraste con las mallas estructuradas donde la vecindad se infiere implícitamente del índice (i,j) (Project Pythia, 2023). Esta flexibilidad permite adaptar la forma y el tamaño de las celdas a la geometría de los datos, eliminando celdas vacías y reduciendo el número total de primitivas geométricas a renderizar.

Los modelos numéricos de próxima generación (MPAS, ICON, CAM-SE) ya adoptan mallas no estructuradas para representar el dominio global con resolución variable (Franda et al., 2023). En el contexto de radar, Kristof (2010) propone que la representación nativa de los datos de radar en mallas no estructuradas reduce tanto el volumen de datos como las distorsiones por interpolación respecto a la conversión forzada a grillas cartesianas.

2.2 Marco Conceptual

Los conceptos clave del presente trabajo se definen a continuación:

Barrido PPI (*Plan Position Indicator*): Exploración en la que la antena mantiene un ángulo de elevación fijo y rota 360° en azimuth. Cada PPI produce una superficie cónica de datos que se proyecta sobre el plano horizontal para su visualización en mapa.

Gate: Celda elemental de medición del radar, definida por la intersección de un ángulo de azimuth y una distancia de rango. Para el KVNx, cada gate cubre 250 m en la dirección radial.

Zarr: Formato de almacenamiento de arrays n-dimensionales basado en chunks comprimidos, diseñado para lectura paralela en entornos de computación en la nube (Zarr Developers, 2024).

ARCO (*Analysis-Ready, Cloud-Optimized*): Paradigma de organización de datos geoespaciales en el que el preprocesamiento, la estandarización de metadatos y el formato de almacenamiento se diseñan conjuntamente para maximizar la accesibilidad y la eficiencia de análisis distribuido (Stern et al., 2021).

Icechunk: Sistema de control de versiones Git-like para *stores* Zarr v3, que garantiza escrituras atómicas y reproducibilidad de los conjuntos de datos (Earthmover, 2024).

Malla no estructurada: Conjunto de nodos con conectividad explícita (lista de triángulos, tetraedros u otros poliedros), cuyas posiciones y vecindades son irregulares (Project Pythia, 2023).

WebGL 2: API de gráficos por GPU disponible en el navegador web, basada en OpenGL ES 3.0, que provee acceso a *instanced rendering*, *Element Buffer Objects* (EBO) y shaders GLSL ES 3.00 (Khronos Group, 2017).

Instanced rendering: Técnica de renderizado GPU en la que una única llamada de dibujo (`drawArraysInstanced`) produce N instancias de la misma geometría base con atributos distintos por instancia, minimizando la sobrecarga de la CPU.

Web Mercator: Proyección cartográfica utilizada por los mapas web modernos (Google Maps, OpenStreetMap, MapLibre). Las coordenadas (longitud, latitud) se mapean a valores en el intervalo $[0, 1]^2$, con el paralelo de Ecuador en $y = 0,5$.

MessagePack: Formato de serialización binaria compacta compatible con JSON, optimizado para la transmisión de arrays numéricos con menor *overhead* que el texto ASCII.

Transferable Objects: Mecanismo de la API de Web Workers que permite mover `ArrayBuffers` entre hilos del navegador en tiempo $O(1)$ sin realizar copias de memoria.

2.3 Marco Espacial

El caso de estudio principal de esta investigación es el radar KVN_X (Vance Air Force Base, Oklahoma, EE.UU.), perteneciente a la red NEXRAD de la NOAA. Sus coordenadas son 36,7408° N, 98,1275° O, con altitud de 408 m s.n.m. El radio de cobertura efectivo en modo vigilancia (VCP-12) es de 460 km para reflectividad y 230 km para velocidad Doppler.

Para el análisis de aplicabilidad en Colombia, se referencian los radares de la red IDEAM, en particular el radar de Carimagua (Meta, Colombia), que opera con el sistema IRIS/Sigmet y cuyos datos son ingresados al *store* ARCO/Zarr mediante `raw2zarr`.

2.4 Marco Temporal

El desarrollo de la herramienta comprende el período 2025–2026. Para la fase de implementación y validación se utiliza un segmento del archivo NEXRAD del radar KVN_X correspondiente al conjunto de 54 volúmenes temporales ($\approx 4,5$ horas de datos), almacenados en el *store* Icechunk KVN_X_IC.zarr.

Conclusiones Parciales

El marco teórico establece tres fundamentos para las decisiones de diseño de la herramienta:

1. La geometría inherentemente cónica de los barridos PPI de radar justifica el uso de mallas no estructuradas: la topología polar de los datos tiene conectividad natural que las grillas cartesianas obligan a destruir mediante interpolación.
2. La combinación Zarr/ARCO + Icechunk (`raw2zarr`) elimina la latencia de acceso a datos que históricamente ha sido el primer cuello de botella; el presente trabajo se enfoca en el segundo cuello de botella: el renderizado.
3. WebAssembly y WebGL 2 proveen la plataforma tecnológica necesaria para alcanzar rendimiento de nivel nativo en el navegador web, sin requerir instalación de software ni acceso a GPUs de alto costo.

3. Arquitectura de Datos

El presente capítulo describe las decisiones de diseño adoptadas para integrar el almacenamiento ARCO/Zarr producido por raw2zarr con el flujo de procesamiento gráfico desarrollado en esta tesis. El objetivo es garantizar que la lectura de los datos sea eficiente, que la estructura original del *store* se preserve sin transformaciones destructivas, y que la interfaz hacia el motor de renderizado esté bien definida.

3.1 Análisis de la Estructura del *Store Icechunk/Zarr*

El *store* KVN_X.IC.zarr generado por raw2zarr adopta la siguiente jerarquía de grupos:

```
KVNX.IC.zarr/  
  VCP-12/  
    sweep_0/      ← elevación 0,48°  
      DBZH        → array (n_times × n_az × n_range)  
      VRADH       → array (n_times × n_az × n_range)  
      ZDR         → array (n_times × n_az × n_range)  
      PHIDP       → array (n_times × n_az × n_range)  
      RHOHV       → array (n_times × n_az × n_range)  
      azimuth     → coordenada (n_times × n_az)  
      range       → coordenada (n_range,)  
      sweep_fixed_angle → coordenada (n_times,)  
    sweep_1/      ← elevación 0,90°  
      ...  
    sweep_16/     ← elevación 19,51°  
      ...
```

Para el conjunto KVN_X VCP-12: $n_times = 54$, $n_az = 720$, $n_range = 1,832$. Cada variable ocupa un único chunk de dimensiones $(1 \times 720 \times 1,832)$, de modo que la lectura de un timestep completo requiere exactamente una operación de I/O por variable.

Esta organización de un chunk por timestep es una elección deliberada de raw2zarr que favorece el patrón de acceso de la visualización: el usuario solicita un instante de tiempo completo, no una pequeña subregión espacial.

3.2 El Problema de la Incompatibilidad de Icechunk con JavaScript

El diseño original contemplaba la lectura directa del *store* Zarr desde el navegador web utilizando bibliotecas de JavaScript como `zarrita.js`. Sin embargo, durante la fase de implementación se identificó una incompatibilidad fundamental: el *store* generado por `raw2zarr` no es un *store* Zarr estándar, sino un *store* gestionado por **Icechunk**, el cual añade una capa de control de versiones con su propio formato de manifiestos y referencias:

```
KVNX_IC.zarr/
  config.yaml      ← metadata Icechunk (protocolo propietario)
  chunks/          ← chunks en formato Icechunk (no Zarr estándar)
  manifests/
  refs/
```

A la fecha de esta investigación (2026), Icechunk no tiene implementación en JavaScript. Las bibliotecas de lectura Zarr para el navegador no pueden acceder a estos almacenes directamente. Esta limitación obligó a redefinir la arquitectura del pipeline de datos.

3.3 Decisión de Diseño: Servidor HTTP como Puente

La solución adoptada introduce un servidor HTTP ligero escrito en Rust que actúa como puente entre el *store* Icechunk y el cliente web:

1. El **servidor Axum** (Tokio Contributors, 2023) inicializa la conexión con el *store* Icechunk
2. El servidor expone tres endpoints HTTP que el navegador puede consumir con peticiones `fetch()` estándar.
3. Los datos se serializan en formato **MessagePack** binario para minimizar el *overhead* de transmisión respecto a JSON: un sweep completo de 1,319,040 gates ocupa $\approx 4,8$ MB en MessagePack frente a ≈ 7 MB en JSON.

Esta arquitectura tiene además una ventaja estratégica para el trabajo futuro: el servidor puede evolucionar hacia un microservicio en la nube sin cambiar la interfaz del *frontend*. El navegador solo necesita conocer la URL del endpoint.

3.4 Soporte Multi-Radar: Arquitectura Genérica

Un requisito fundamental del diseño es que el pipeline no esté atado a un único radar o formato de datos. La validación sobre el radar Guaviare del IDEAM (San José del Guaviare, Colombia,

formato IRIS/Sigmat) demostró que los *stores* generados por raw2zarr con distintos formatos origen presentan diferencias estructurales relevantes que el servidor debe absorber.

La Tabla 3.1 compara las estructuras de los *stores* KVN_X (NEXRAD) y Guaviare (IRIS/Sigmat).

Cuadro 3.1: Comparación de estructuras del store: KVN_X (NEXRAD) vs. Guaviare (IRIS/Sigmat)

Aspecto	KVN _X (NEXRAD)	Guaviare (IRIS/Sigmat)
Grupos raíz (VCPs)	VCP-12	SURVP, PRECA, PRECB, PRECC
Azimuths por sweep	720 (paso 0,5°)	360 (paso 1°)
Range gates	1,832	747 (PRECA/sweep_0)
Gate size	250 m	300 m
Variable velocidad	VRADH	WRADH
Variable correlación	RhoHV	RHOHV
Coordenadas	36,74° N, 98,13° O	2,53° N, 72,63° O
Sweeps por VCP	17 (VCP-12)	4 (PRECA)

Para gestionar esta heterogeneidad, el servidor mantiene un *catálogo de radares* (RadarConfig) con metadatos de cada sitio, y un AppState que indexa un store Icechunk abierto por cada radar disponible. El endpoint `/api/radars` expone el catálogo al *frontend* para construir el selector de radar en la interfaz. Las coordenadas del radar y el tamaño del gate se leen dinámicamente del *store* en lugar de estar codificadas de forma estática, lo que garantiza la corrección geográfica sin modificaciones para nuevos radares.

La Figura 3.1 muestra la reflectividad (DBZH) del radar Guaviare renderizada sobre el mapa de Colombia, evidenciando la correcta georreferenciación del PPI sobre el departamento del Guaviare.

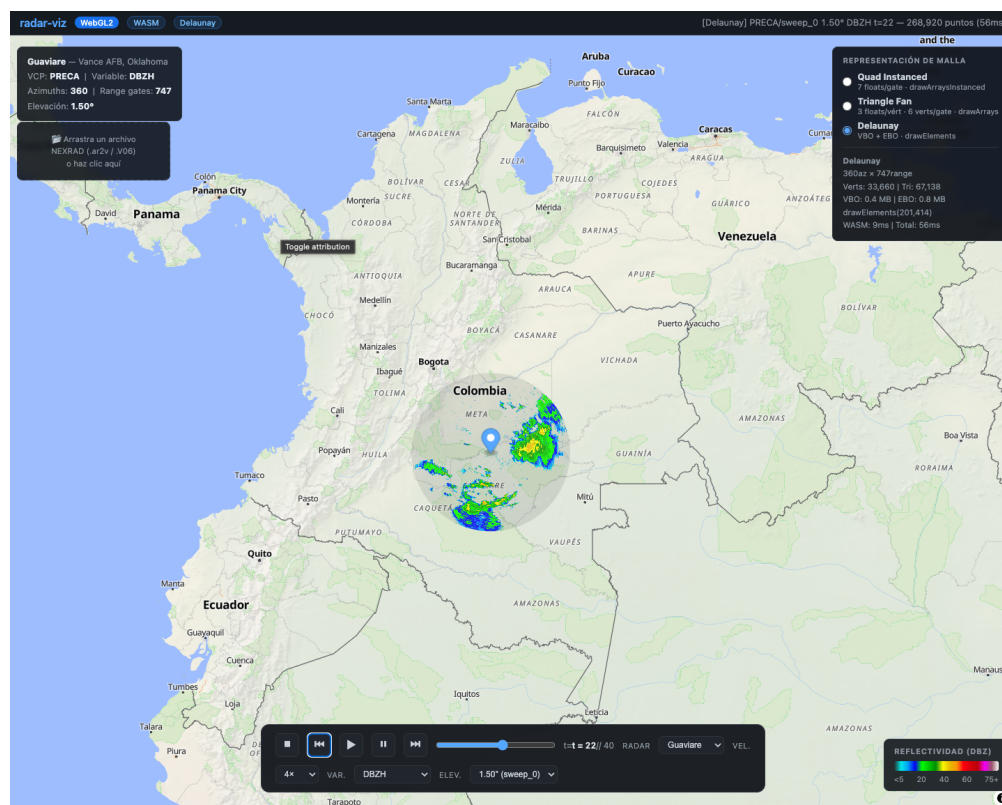


Figura 3.1: *PPI de reflectividad (DBZH) del radar Guaviare (IDEAM, Colombia) renderizado con la herramienta desarrollada. El radar se ubica en San José del Guaviare ($2,53^{\circ}$ N, $72,63^{\circ}$ O); el VCP es PRECA, sweep_0 a elevación mínima. Los valores se muestran en la misma escala de colores NWS que para KVNx, sin cambios en el motor de renderizado.*

3.5 API del Servidor

El servidor implementa los endpoints documentados en la Tabla 3.2. Todos los endpoints que acceden a datos de un radar específico incluyen el parámetro {radar} como primer segmento de la ruta, permitiendo distinguir entre múltiples stores abiertos simultáneamente.

Cuadro 3.2: *Endpoints del servidor Axum de radar-viz*

Endpoint	Formato	Descripción
GET /health	texto	Verificación de disponibilidad del servidor
GET /api/radars	JSON	Catálogo de radares disponibles: id, nombre, coordenadas, VCP por defecto
GET /api/meta/{radar}/{vcp}	JSON	Metadatos del VCP: número de timesteps y sweeps
GET /api/sweep/{radar}/{vcp}/{sweep}/{var}?time=N	MessagePack	Datos completos de un sweep: azimuth, range, data, elevación, posición del radar

La respuesta del endpoint principal contiene la estructura `SweepResponse`, definida en Rust como:

```
pub struct SweepResponse {
    pub azimuth: Vec<f64>, // N_az ángulos en grados [0°, 360°)
    pub range: Vec<f32>, // N_range distancias en metros
    pub data: Vec<f32>, // N_az × N_range, row-major, NaN = sin dato
    pub elevation: f32, // ángulo de elevación del sweep
    pub radar_lon: f64, // longitud decimal del radar
    pub radar_lat: f64, // latitud decimal del radar
    pub gate_size: f64, // tamaño del gate en metros (250,0)
    pub n_az: usize,
    pub n_range: usize,
}
```

La serialización se realiza con `rpm_serde::to_vec_named()`, que produce un mapa `MessagePack` con claves de tipo cadena, necesario para que la biblioteca `@msgpack/msgpack` en JavaScript deserialice los campos correctamente por nombre. Una trampa común al cruzar la frontera Rust/JavaScript es usar `to_vec()` en lugar de `to_vec_named()`: el primero serializa los campos como un array posicional, haciendo que el decoder JavaScript asigne claves numéricas (“0”, “1”, ...) y devuelva `undefined` para todos los accesos por nombre.

3.6 Rendimiento del Pipeline de Datos

La Tabla 3.3 reporta los tiempos medidos del servidor sobre el hardware de referencia (Apple M4 Pro, almacenamiento SSD NVMe local).

Cuadro 3.3: *Tiempos medidos del servidor Axum, sweep_0 DBZH, KVNK VCP-12*

Operación	Tiempo típico
Apertura del <i>Store</i> Icechunk (inicio)	≈ 200 ms
Lectura de chunk Zarr (sweep_0, 5,3 MB)	150 – 300 ms
Serialización MessagePack	30 – 50 ms
Transferencia HTTP (loopback)	< 5 ms
Total por solicitud (sweep_0)	≈ 250 ms

El cuello de botella es la lectura del chunk desde el SSD. Las solicitudes subsiguientes del mismo sweep en una sesión activa se benefician del caché del sistema operativo y toman del orden de 100 ms. En un escenario de producción con el *store* alojado en almacenamiento en la nube (S3, GCS), el tiempo dependería del ancho de banda disponible.

Para comparación, la lectura de un archivo NEXRAD Level II equivalente con Py-ART toma $\bar{x} = 1,326$ ms (medición propia, Sección 7.2), lo que representa una reducción de latencia de un factor 5,3× en la fase de acceso a datos.

3.7 Flujo Completo: del *Store* Zarr a la GPU

El flujo de procesamiento gráfico completo, desde el *store* hasta el primer píxel renderizado, se resume en la Figura 3.2 y consta de las siguientes etapas:

1. **Lectura desde Icechunk:** el servidor en Rust accede al *store* y recupera el *chunk* correspondiente mediante `array.async_retrieve_chunk([t, 0, 0])`.
2. **Serialización MessagePack:** la estructura `SweepResponse` se serializa a bytes y se envía al navegador como respuesta HTTP.
3. **Fetch + decode en Worker I/O:** un Web Worker del navegador realiza la solicitud HTTP, decodifica MessagePack con `@msgpack/msgpack` y convierte los arrays a `TypedArrays` (`Float32Array`, `Float64Array`).
4. **Transformación WASM en Worker Vertices:** los `TypedArrays` se transfieren (sin copia) al Worker Vertices, que llama a la función `sweep_to_vertices()` compilada desde Rust. Esta función genera el buffer de vértices para la GPU.

5. **Carga a GPU y renderizado:** el buffer resultante se envía (sin copia) al hilo principal y se sube a la memoria GPU mediante `gl.bufferData()`.

Las etapas 3, 4 y 5 operan en hilos separados del navegador (Web Workers), lo que garantiza que el mapa base continúe respondiendo a interacciones del usuario (pan, zoom) durante toda la carga. La arquitectura de Workers se describe en detalle en el Capítulo 5.

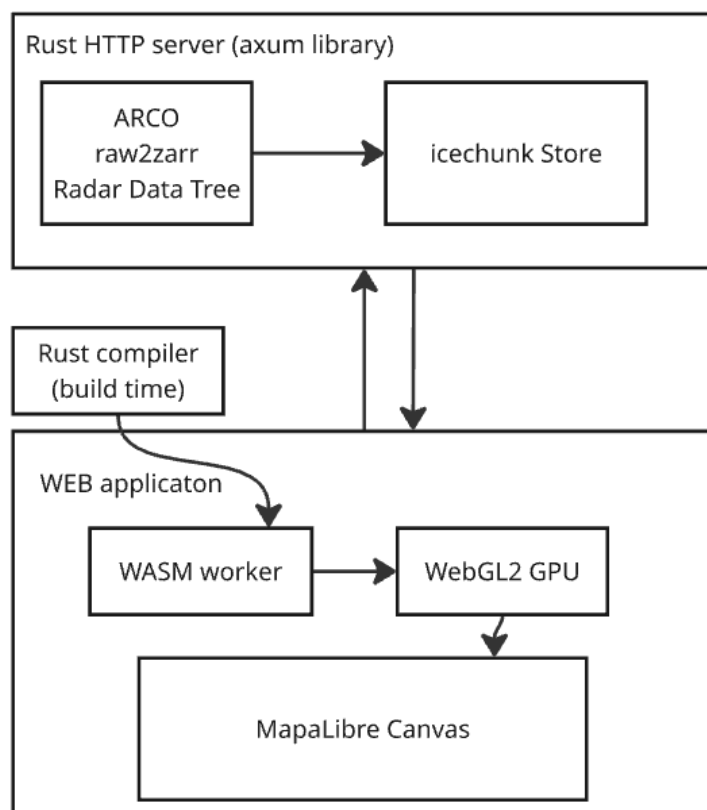


Figura 3.2: *Arquitectura del pipeline de datos: el servidor Axum (Rust) actúa como puente entre el Store Icechunk/Zarr y la aplicación web. El módulo WASM transforma los datos en vértices GPU dentro de un Web Worker, dejando el hilo principal libre para el renderizado interactivo.*

Conclusiones Parciales

La arquitectura de datos adoptada resuelve la incompatibilidad entre Icechunk y el cliente web mediante un servidor HTTP ligero en Rust que preserva todas las propiedades de lectura eficiente del *store* ARCO/Zarr. La organización del *store* (un *chunk* por paso temporal y variable) se alinea naturalmente con el patrón de acceso de la visualización interactiva. La serialización mediante MessagePack reduce el tamaño de transferencia en comparación con JSON y minimiza el tiempo de decodificación en JavaScript frente al costoso análisis sintáctico de texto plano. La reducción de

latencia de acceso frente a Py-ART (5,3×) valida la hipótesis de `raw2zarr` y desplaza el cuello de botella computacional hacia el renderizado, desafío que se aborda exhaustivamente en los capítulos siguientes.

4. Mallas No Estructuradas

El título de esta tesis hace referencia explícita a las *mallas no estructuradas* como herramienta central para la representación de datos de radar. Este capítulo justifica esa elección, describe las tres estrategias de malla diseñadas e implementadas, detalla la matemática de cada una y presenta las implicaciones prácticas de cada enfoque.

4.1 Motivación: La Geometría Polar del Radar

Cada gate de un barrido PPI ocupa un sector de anillo polar cuyos lados son dos radios (bordes interior y exterior, separados por el tamaño del gate $\delta r = 250$ m) y dos arcos circulares (bordes azimutal izquierdo y derecho, separados por el paso angular $\delta\phi$). A diferencia de una grilla cartesiana donde todas las celdas son congruentes, este sector es un cuadrilátero *no rectangular*: su dimensión azimutal crece linealmente con el rango:

$$w_{az}(r) = r_{suelo} \cdot \delta\phi_{rad} \quad (4.1)$$

donde $r_{suelo} = r_{slant} \cos(\theta_{el})$ es el rango proyectado sobre el suelo y $\delta\phi_{rad}$ es el paso azimutal en radianes. Para el VCP-12 con 720 azimuths ($\delta\phi = 0,5^\circ = 8,73 \times 10^{-3}$ rad), el ancho azimutal oscila entre ≈ 870 m al primer gate (100 km de rango) y $\approx 2,100$ m al último gate (240 km). Representar estos gates como cuadrados uniformes en una grilla cartesiana introduce errores geométricos que crecen con la distancia al radar (Kristof, 2010).

4.2 Las Tres Estrategias de Malla Implementadas

Esta investigación implementa tres estrategias de representación geométrica, cada una con diferentes características de fidelidad, tamaño de buffer y complejidad computacional.

4.2.1 Representación A: Quads Instanciados (*Quad Instanced*) La primera estrategia representa cada gate como una instancia de un quad unitario $[-0,5, +0,5]^2$ transformado mediante una **matriz afín 2×3** que codifica su posición, orientación y dimensiones exactas en coordenadas Web Mercator.

Formulación matemática Para un gate en azimuth ϕ (en grados, convención meteorológica Norte- 0°) y rango oblicuo r_{slant} , la matriz afín se calcula como sigue. Sean:

$$\phi_{\text{rad}} = \phi \cdot \frac{\pi}{180} \quad (4.2)$$

$$\theta_{\text{el,rad}} = \theta_{\text{el}} \cdot \frac{\pi}{180} \quad (4.3)$$

$$r_{\text{suelo}} = r_{\text{slant}} \cdot \cos(\theta_{\text{el,rad}}) \quad (4.4)$$

$$s_{\text{Mercator}} = \frac{1}{C_{\text{Tierra}} \cos(\phi_{\text{lat,radar}})} \quad (4.5)$$

donde $C_{\text{Tierra}} = 40,075,016,686$ m es la circunferencia ecuatorial y $\phi_{\text{lat,radar}}$ es la latitud del radar. Entonces:

$$s_r = \delta r \cdot s_{\text{Mercator}} \quad (\text{escala radial, constante}) \quad (4.6)$$

$$s_\phi = r_{\text{suelo}} \cdot \delta \phi_{\text{rad}} \cdot s_{\text{Mercator}} \quad (\text{escala azimutal, variable con } r) \quad (4.7)$$

Las columnas de la matriz afín son:

$$\mathbf{c}_1 = (\sin \phi_{\text{rad}}, -\cos \phi_{\text{rad}}) \cdot s_r \quad (\text{vector radial}) \quad (4.8)$$

$$\mathbf{c}_2 = (\cos \phi_{\text{rad}}, \sin \phi_{\text{rad}}) \cdot s_\phi \quad (\text{vector azimutal}) \quad (4.9)$$

Esta formulación garantiza que los quads adyacentes en rango se toquen sin huecos (el paso radial s_r es exactamente el ancho del gate) y que los quads adyacentes en azimuth se toquen sin huecos (el paso azimutal s_ϕ es exactamente el arco entre azimuths consecutivos a esa distancia).

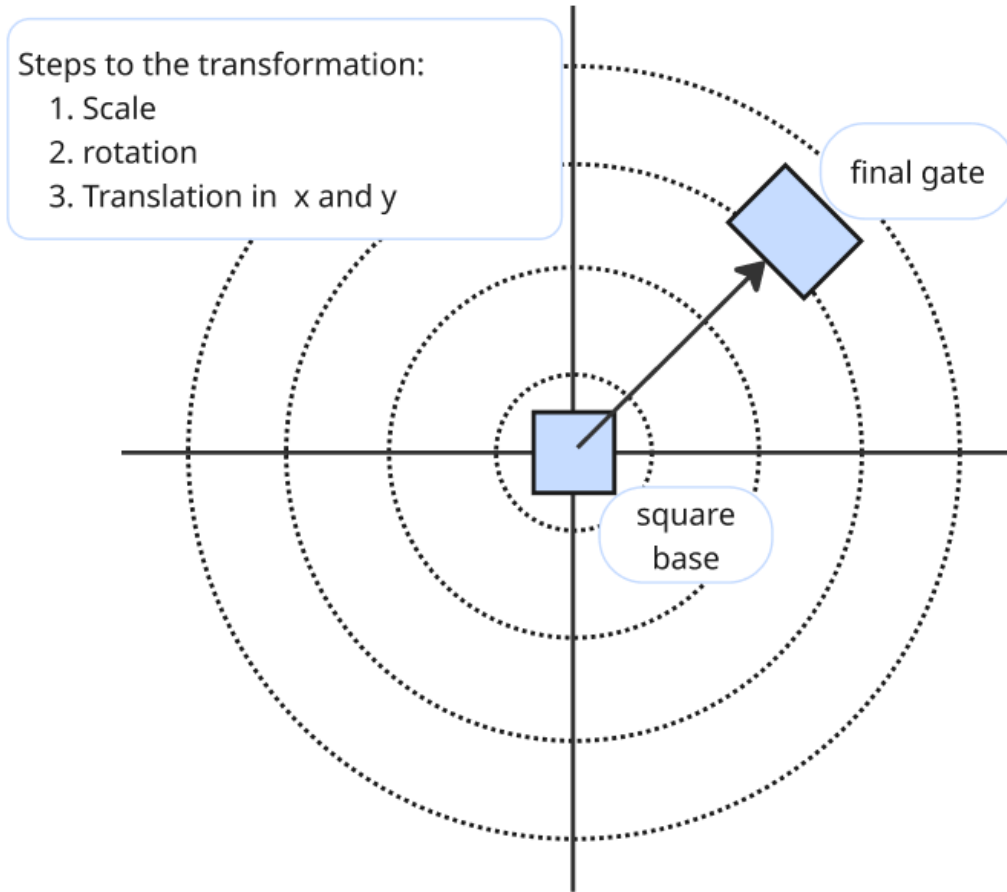
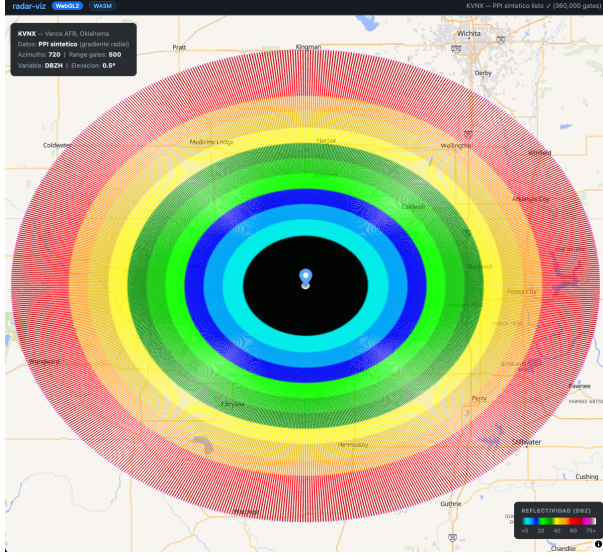
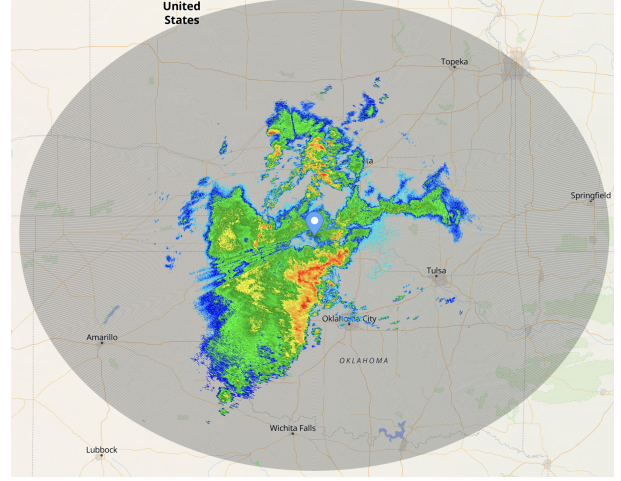


Figura 4.1: Pipeline de transformación afín para un gate de radar en la Representación A. Un quad unitario es: (1) escalado con s_r en la dirección radial y s_ϕ en la azimutal, (2) rotado según el azimuth ϕ_i , y (3) trasladado al centro del gate $\mathbf{p}_{ij}$ en coordenadas Web Mercator. Los tres pasos corresponden directamente a las columnas de la matriz $\mathbf{M}_{ij}$ (Ec. 4.11).

Corrección del error geométrico original La validación del renderer se realizó en dos fases. En la primera fase, el pipeline completo: colormap NWS, textura WebGL de 256 píxeles, instanced rendering y la integración con MapLibre, se verificó sobre un PPI sintético de gradiente radial: $\text{dBZ}(r) = -10 + (r/N_{\text{range}}) \times 75$, con 720 azimuths y 500 range gates a una elevación de $0,5^\circ$. La Figura 4.2(a) muestra el resultado de esta primera renderización: el colormap NWS reproduce correctamente la escala cromática estándar del NWS (de azul-cyan para valores bajos a rojo-blanco para valores altos) y la proyección sobre el mapa MapLibre es geográficamente correcta. El alcance reducido ($500 \text{ gates} \times 250 \text{ m} = 125 \text{ km}$) se eligió deliberadamente para acelerar la compilación y verificar visualmente cada gate de forma individual.



(a) datos sintéticos, 720×500 gates, colormap NWS validado. Escala uniforme w aplicada a ambas columnas de la matriz afín.



(b) Estado final: datos reales KVNK, 720×1,832 gates, escalas s_r y s_ϕ independientes. Sin huecos a ningún rango.

Figura 4.2: Evolución de la Representación A: de la primera renderización con datos sintéticos (a) a la versión corregida con datos reales (b). La diferencia en el relleno de los anillos exteriores evidencia el efecto de la corrección de escala.

Al pasar a datos reales del radar KVNK con 1,832 range gates (alcance de 458 km), se identificó un error geométrico sistemático en la implementación inicial: ambas columnas de la matriz afín usaban la misma escala escalar w , que codificaba únicamente el tamaño del gate en la dirección radial:

$$\mathbf{M}_{\text{incorrecto}} = \begin{pmatrix} \sin \phi \cdot w & \cos \phi \cdot w \\ -\cos \phi \cdot w & \sin \phi \cdot w \end{pmatrix} \quad (4.10)$$

Esta formulación trata el gate como un cuadrado en el espacio local del quad ($w \times w$), lo que equivale a asumir que el ancho azimuthal de un gate es igual a su profundidad radial. Esta suposición es válida solo cuando $r_{\text{suelo}} \cdot \delta\phi_{\text{rad}} \approx \delta r$, lo que ocurre para el primer gate del VCP-12 ($r \approx 2,125$ m) pero produce un error creciente con el rango: a 240 km, el ancho azimuthal real es $\approx 2,100$ m mientras que $\delta r = 250$ m, un factor 8,4 de diferencia. El resultado visible son huecos angulares entre rayos que se ensanchan proporcionalmente al rango.

La corrección consistió en separar explícitamente las dos escalas según la ecuación 4.1: la escala radial $s_r = \delta r \cdot s_{\text{Mercator}}$ es constante para todo el sweep, mientras que la escala azimuthal $s_\phi = r_{\text{suelo}} \cdot \delta\phi_{\text{rad}} \cdot s_{\text{Mercator}}$ se recalcula para cada gate en función de su rango proyectado sobre el suelo. La matriz corregida queda:

$$\mathbf{M}_{\text{correcto}} = \begin{pmatrix} \sin \phi \cdot s_r & \cos \phi \cdot s_\phi \\ -\cos \phi \cdot s_r & \sin \phi \cdot s_\phi \end{pmatrix} \quad (4.11)$$

Con esta corrección, los quads adyacentes en azimuth se tocan exactamente sin huecos ni solapamientos a cualquier rango, y la Figura 4.2(b) muestra el resultado sobre el radar KVNx: un PPI de reflectividad real con cobertura continua desde el primer gate hasta los 458 km de alcance máximo.

Layout del buffer El buffer de la Representación A tiene exactamente $N_{az} \times N_{range} \times 7$ elementos de tipo float32. Cada gate ocupa 7 floats contiguos (28 bytes), con el layout descrito en la Tabla 4.1.

Cuadro 4.1: *Layout del buffer de vértices para Quads Instanciados (7 floats/gate)*

Offset (floats)	Campo	Descripción
0	row1_x	Componente x de la columna radial
1	row1_y	Componente y de la columna radial
2	translate_x	Posición Mercator x del centro del gate
3	row2_x	Componente x de la columna azimuthal
4	row2_y	Componente y de la columna azimuthal
5	translate_y	Posición Mercator y del centro del gate
6	value	Valor del dato radar (dBZ, m/s, etc.)

Para el sweep_0 de KVNx: $720 \times 1,832 \times 7 \times 4 \text{ bytes} = 36,9 \text{ MB}$ de buffer.

4.2.2 Representación B: Triangle Fan (Vértices Explícitos) La segunda estrategia calcula explícitamente las 4 esquinas de cada gate en coordenadas Mercator y las organiza como 2 triángulos con vértices explícitos (no comprimidos mediante instancing). El objetivo es producir vértices directamente integrables con herramientas GIS y con otras bibliotecas geométricas que operan sobre coordenadas cartesianas.

Formulación matemática Para el gate (i_{az}, i_r) , se calculan las 4 esquinas del sector polar:

$$r_{\text{cerca}} = (r_{\text{slant}} - \delta r / 2) \cos \theta_{\text{el}} \quad (4.12)$$

$$r_{\text{lejos}} = (r_{\text{slant}} + \delta r / 2) \cos \theta_{\text{el}} \quad (4.13)$$

$$\phi_{\text{iz}} = \phi_{\text{rad}} - \delta \phi_{\text{rad}} / 2 \quad (4.14)$$

$$\phi_{\text{de}} = \phi_{\text{rad}} + \delta \phi_{\text{rad}} / 2 \quad (4.15)$$

Las 4 esquinas en Mercator son:

TC (*Top-Close*): borde interior, arco izquierdo: $(x_{\text{TC}}, y_{\text{TC}})$

TD (*Top-Der*): borde interior, arco derecho

BC (*Bot-Close*): borde exterior, arco izquierdo

BD (*Bot-Der*): borde exterior, arco derecho

Los 2 triángulos son: $\text{TC} \rightarrow \text{TD} \rightarrow \text{BC}$ y $\text{TD} \rightarrow \text{BD} \rightarrow \text{BC}$. Cada vértice almacena 3 floats: $(x_{\text{Mercator}}, y_{\text{Mercator}}, \text{valor})$.

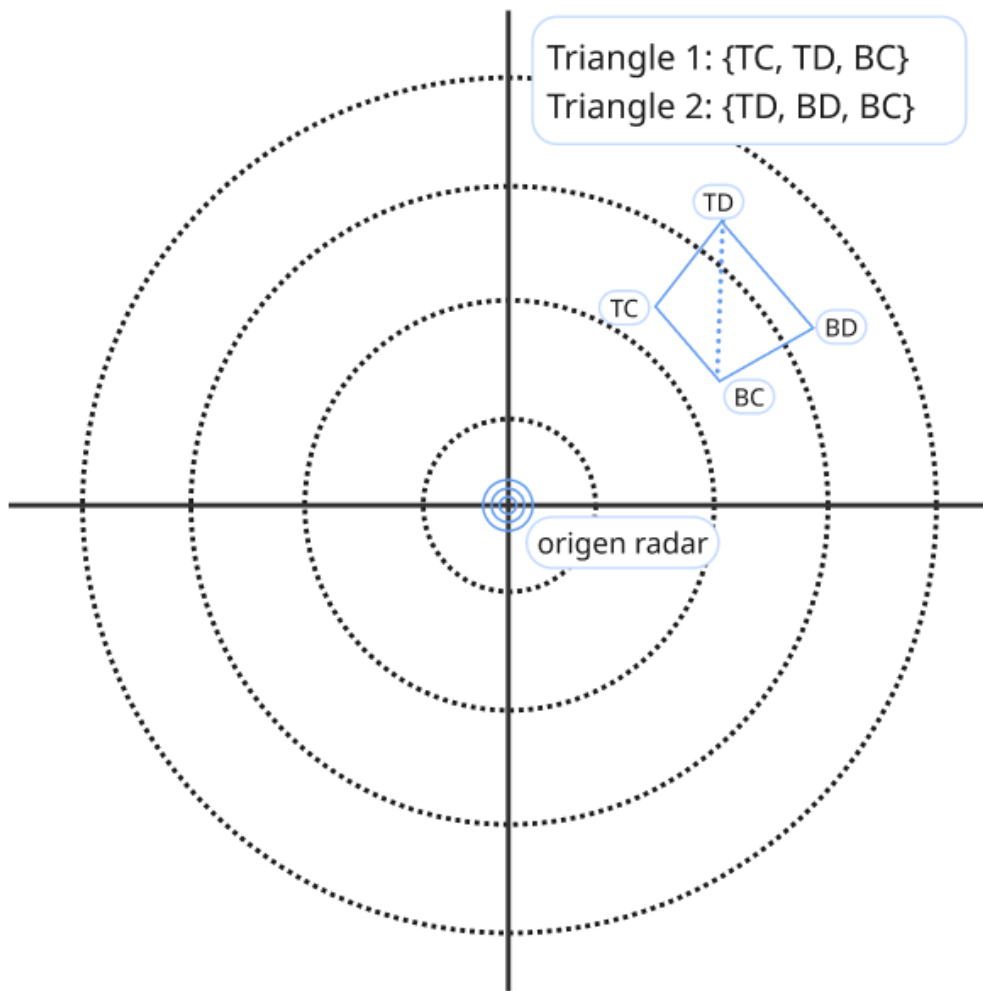


Figura 4.3: Geometría de la Representación B para un gate. Las cuatro esquinas del sector polar (TC, TD, BC, BD) se calculan explícitamente en coordenadas Web Mercator y se organizan en dos triángulos: $TC \rightarrow TD \rightarrow BC$ y $TD \rightarrow BD \rightarrow BC$. La diagonal compartida $TD \rightarrow BC$ es visible como arista interna en zoom extremo.

Layout del buffer El buffer tiene $N_{az} \times N_{range} \times 18$ floats: 2 triángulos $\times$ 3 vértices $\times$ 3 floats. Para sweep_0 de KVNx: $720 \times 1,832 \times 18 \times 4$ bytes = 95,2 MB, significativamente mayor que la Representación A.

4.2.3 Representación C: Triangulación de Delaunay La tercera estrategia aplica la triangulación de Delaunay (Delaunay, 1934) sobre el conjunto de centros de gate en coordenadas Mercator. La triangulación de Delaunay produce una malla que maximiza el ángulo mínimo de todos los triángulos, evitando triángulos muy alargados que producirían artefactos de interpolación (Kristof, 2010).

Algoritmo

1. Calcular el centroide Mercator de cada uno de los $N_{az} \times N_{range}$ gates (idéntico al centro del quad de la Representación A).
2. Pasar los N puntos a la crate `delaunator` (algoritmo de Bowyer-Watson incremental), que devuelve una lista de tripletas de índices.
3. Construir el buffer de vértices (3 floats por punto) y el *Element Buffer Object* (EBO) con los índices u32.

La crate `delaunator` es Rust puro (sin dependencias C, sin threads), por lo que compila también para `wasm32-unknown-unknown`, permitiendo ejecutar la triangulación en el mismo módulo WASM que las demás transformaciones.

Layout del buffer La Representación C produce *dos* buffers:

- **Buffer de vértices:** $N \times 3$ floats ($N = N_{az} \times N_{range} = 1,319,040$ para `sweep_0`). Tamaño: $1,319,040 \times 3 \times 4$ bytes = 15,8 MB.
- **EBO (índices):** $\approx 2N \times 3$ uint32 = $2,638,080 \times 3$ índices. Tamaño: ≈ 30 MB.

La llamada de dibujo usa `drawElements(TRIANGLES, N_idx, UNSIGNED_INT, 0)`, aprovechando que los vértices no se duplican entre triángulos adyacentes.

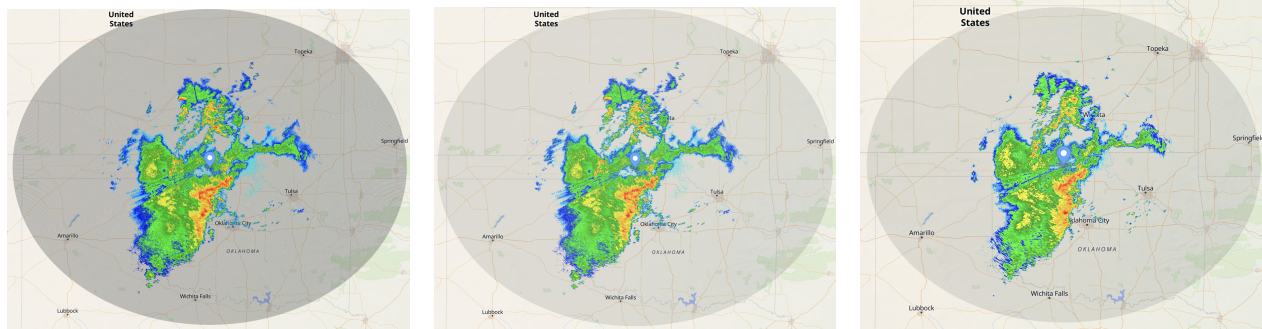
4.3 Comparación de las Tres Representaciones

La Tabla 4.2 consolida las características clave de las tres representaciones, con datos medidos sobre `sweep_0 DBZH` de KVN_X (1,319,040 gates) en Chrome 125 sobre Apple M4 Pro.

Cuadro 4.2: Comparación de estrategias de malla, *sweep_0 KVN_X, 1,319,040 gates*

Métrica	Quad (A)	Triangle Fan (B)	Delaunay (C)
Floats en buffer	9,233,280	23,742,720	3,957,120
Tamaño buffer (MB)	35,3	90,7	15,1 (+ 10 MB EBO)
Tiempo de cómputo	13	63	89
WASM (ms)			
Primitiva GPU	TRIANGLES inst.	TRIANGLES	TRIANGLES (EBO)
Llamada de dibujo	<code>drawArraysInstanced</code>	<code>drawArrays</code>	<code>drawElements</code>
Duplica vértices	No (instancing)	Sí (×6)	No (EBO)
Complejidad Rust	$O(N)$	$O(N)$	$O(N \log N)$
Compatibilidad WASM	✓	✓	✓

Los tiempos de la columna “Tiempo de cómputo WASM” corresponden a ejecuciones sin fetch (datos ya en caché), medidos mediante `performance.now()` en el Worker Vertices. Las mediciones se realizaron en modo `RECOMPUTE_VERTICES`, que reutiliza los datos del caché sin re-descargar desde el servidor.



(a) **Rep. A: Quad Instanciado.** 7 floats/gate · 6 vértices/gate · drawArrays. VBO: drawArraysInstanced. VBO: 95,2 MB; WASM: 120 ms. 35,3 MB; WASM: 50 ms.

(b) **Rep. B: Triangle Fan.** 6 vértices/gate · drawArrays. VBO: drawArraysInstanced. VBO: 95,2 MB; WASM: 120 ms. 35,3 MB; WASM: 50 ms.

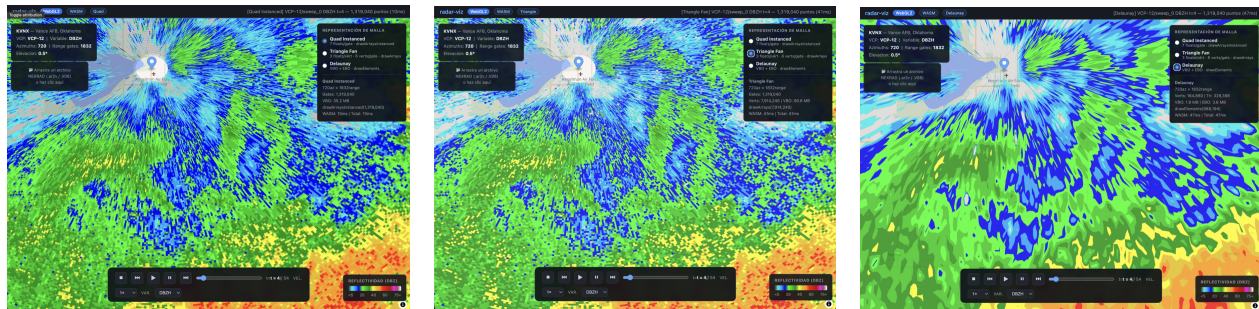
(c) **Rep. C: Delaunay Submuestreado.** $N' = 164,880$ pts · drawElements. VBO+EBO: 5,7 MB; WASM: 70 ms.

Figura 4.4: Barrido KVN sweep_0 (DBZH, $\varphi = 0,48^\circ$, VCP-12) renderizado con las tres representaciones de malla a zoom nivel 7 (alcance completo de 458 km). Las diferencias visuales son imperceptibles a este nivel de zoom; emergen únicamente en zoom extremo (Figura 4.5).

4.3.1 Análisis y Elección de Representación para Tiempo Real Los resultados de la Tabla 4.2 permiten extraer las siguientes conclusiones:

- **Quad Instanciado (A)** ofrece el menor tiempo de cómputo WASM (13 ms frente a 63 ms y 89 ms de las alternativas) y el buffer de mayor tamaño moderado (35 MB). La técnica de *instanced rendering* delega a la GPU la transformación de cada quad, reduciendo la carga de CPU. Es la estrategia óptima para el caso de uso de visualización en tiempo casi real.
- **Triangle Fan (B)** produce el buffer más grande (90 MB) porque duplica las coordenadas Mercator de los 6 vértices de cada gate. Sin embargo, estos vértices explícitos son directamente consumibles por herramientas GIS externas (shapefiles, GeoJSON, CGAL), haciendo de esta representación la opción más interoperable.
- **Delaunay (C)** produce el buffer de vértices más compacto (15 MB) y la malla de mayor calidad geométrica (ángulo mínimo maximizado), a costa del mayor tiempo de cómputo (89 ms, dominado por el algoritmo de triangulación). Es la representación más adecuada para análisis científico donde la fidelidad de interpolación es crítica.

La herramienta desarrollada implementa las tres estrategias y permite alternar entre ellas en tiempo de ejecución sin recargar datos desde el servidor, lo que facilita la comparación interactiva.



(a) **Rep. A** (zoom extremo). Quads contiguos sin huecos azimutales gracias a la corrección de escalas independientes (Ec. 4.11). WASM: 10 ms.

(b) **Rep. B** (zoom extremo). Esquinas exactas del gate; la diagonal de teselación (TC→TD→BC) es visible como arista interna. WASM: 41 ms.

(c) **Rep. C** (zoom extremo). Triángulos Delaunay sobre centroides submuestreados 8×; los bordes no se alinean con los bordes polares del gate. WASM: 47 ms.

Figura 4.5: Comparación en zoom extremo de las tres representaciones (misma sesión, KVNx DBZH). Las diferencias geométricas entre Rep. A y Rep. B son sub-píxel a este zoom; Rep. C muestra el suavizado del submuestreo 8×.

Conclusiones Parciales

Este capítulo demostró que la geometría polar de los barridos PPI de radar justifica formalmente el uso de mallas no estructuradas: las dimensiones radial y azimutal de cada gate son heterogéneas y solo la estrategia de malla que respeta esta heterogeneidad (escalas s_r y s_ϕ independientes) produce una representación sin huecos ni solapamientos. Las tres estrategias implementadas ofrecen diferentes equilibrios entre rendimiento (Quad A), interoperabilidad (Triangle B) y calidad geométrica (Delaunay C). Para el caso de uso central de esta tesis (visualización interactiva en tiempo real) la Representación A es la óptima, con 13 ms de cómputo para 1,3 millones de gates.

5. Renderizado Interactivo

El renderizado interactivo es el componente de la herramienta que transforma el buffer de vértices generado por la función WASM en píxeles visibles sobre un mapa geográfico, manteniendo una tasa de refresco suficiente para la exploración fluida. Este capítulo describe la arquitectura de procesamiento paralelo basada en Web Workers, los shaders GLSL que implementan el pipeline de rasterización, y la integración con MapLibre GL JS.

5.1 Arquitectura de Procesamiento Paralelo: Web Workers

Un barrido de radar completo involucra tres operaciones costosas que, si se ejecutan en el hilo principal del navegador, bloquean el *event loop* y congelan la interfaz:

Cuadro 5.1: Operaciones costosas y su impacto si ocurren en el hilo principal

Operación	Tiempo típico	Efecto en hilo principal
Fetch HTTP + lectura del body (≈ 5 MB)	350 – 450 ms	Bloquea el event loop
Decodificación MessagePack	30 – 60 ms	Bloquea el event loop
sweep_to_vertices() WASM	13 – 89 ms	Bloquea el event loop

La solución adoptada delega las tres operaciones a **Web Workers**: hilos del sistema operativo independientes del hilo principal del navegador, que se comunican mediante mensajes asíncronos. La arquitectura implementada sigue el patrón I/O Worker + Compute Worker identificado en el análisis de herramientas de referencia:

Worker I/O (worker-io.js): responsable del acceso a red: realiza el `fetch()` al servidor Axum, decodifica MessagePack y coordina el pipeline delegando al Worker Vertices.

Worker Vertices (worker-vertices.js): responsable del cómputo geométrico: inicializa el módulo WASM (`radar_viz_bg.wasm`, 31 KB), recibe los TypedArrays del Worker I/O y ejecuta `sweep_to_vertices()` (o la función de malla seleccionada).

5.1.1 Protocolo de Mensajes Todos los mensajes entre hilos siguen el formato `{type: string, ...payload}`. La secuencia de inicialización es la siguiente:

1. El hilo principal crea el Worker Vertices. Este carga inmediatamente el módulo WASM y, al completar, emite `WASM_READY`.
2. El hilo principal recibe `WASM_READY` y crea el Worker I/O, enviándole `INIT` con la URL del Worker Vertices.
3. El Worker I/O instancia su propia copia interna del Worker Vertices, espera a `WASM_READY`, y emite `READY` al hilo principal.
4. A partir de este punto, el hilo principal puede enviar `LOAD_SWEEP` al Worker I/O para cualquier combinación de VCP, sweep, variable y timestep.

El Worker I/O no puede reutilizar el Worker Vertices del hilo principal porque los handles de Worker no son transferibles entre hilos. La solución es que el Worker I/O instancie su propia copia: el costo es mínimo (31 KB de WASM ya disponible en caché del navegador).

5.1.2 Transferable Objects y Eficiencia de Memoria Un aspecto crítico del protocolo es el uso de **Transferable Objects**: los `ArrayBuffers` se *mueven* (en tiempo $O(1)$) entre Workers en lugar de copiarse. Para un sweep de KVN sweep_0:

Cuadro 5.2: Ahorro de memoria mediante *Transferable Objects*

Buffer	Elementos	Tamaño	Costo (Transferable)
azimuth	720	5,76 KB	$O(1)$
range	1,832	7,33 KB	$O(1)$
data	1,319,040	5,02 MB	$O(1)$
vertexBuffer	9,233,280	35,2 MB	$O(1)$

Sin Transferables, cada carga requeriría copiar ≈ 40 MB entre Workers en un ciclo request. Con Transferables, la transferencia total suma microsegundos independientemente del tamaño.

5.1.3 Resolución de la Condición de Carrera en el Arranque Durante el desarrollo se identificó una condición de carrera: el Worker I/O emitía `READY` inmediatamente después de crear el Worker Vertices interno, antes de que el módulo WASM finalizara su compilación. El hilo principal respondía con `LOAD_SWEEP` y el Worker I/O ejecutaba el fetch para enviar `COMPUTE.VERTICALS`. No obstante, en este punto el Worker Vertices aún carecía del binario WASM disponible para procesar la petición.

La solución fue un **relay selectivo**: el Worker I/O intercepta `WASM_READY` del Worker Vertices interno y lo convierte en `READY` hacia el hilo principal, garantizando que el pipeline no arranque hasta

que WASM esté efectivamente compilado. En el Worker Vertices, el polling con `setInterval` fue reemplazado por `await wasmReadyPromise`, que resuelve cuando `init()` completa.

5.2 Módulo WASM: Compilación Rust → WebAssembly

El módulo de transformación geométrica se compila desde Rust usando `wasm-bindgen` (The Rust and WebAssembly Working Group, 2023) y `wasm-pack`. La crate `radar-viz` se compila de forma condicional: las dependencias incompatibles con `wasm32` (`tokio`, `zarrs`, `icechunk`) se declaran en secciones `[target.'cfg(not(target_arch = "wasm32"))'.dependencies]`, lo que permite producir dos binarios desde el mismo código fuente: el ejecutable nativo del servidor Axum y el módulo WASM del navegador.

El módulo WASM compilado en modo `--release` tiene un tamaño de 31 KB (`radar_viz.bg.wasm` + 14 KB de glue code JavaScript), porque contiene exclusivamente matemática pura sin las dependencias de I/O.

Las funciones públicas del módulo WASM se listan en la Tabla 5.3.

Cuadro 5.3: API pública del módulo WASM

Función	Descripción
<code>sweep_to_vertices(az, range, data, elev, lon, lat, gate_size)</code>	Genera buffer de Quads Instanciados. Retorna <code>Float32Array</code> de $N \times 7$ floats.
<code>sweep_to_triangles(az, range, data, elev, lon, lat, gate_size)</code>	Genera buffer de Triangle Fan. Retorna <code>Float32Array</code> de $N \times 18$ floats.
<code>sweep_to_delaunay(az, range, data, elev, lon, lat, gate_size)</code>	Genera malla de Delaunay. Retorna objeto <code>WasmDelaunayMesh</code> con <code>vertices()</code> e <code>indices()</code> .
<code>lnglat_to_mercator(lon, lat)</code>	Convierte coordenadas geográficas a Web Mercator.
<code>gate_vertex_floats() / gate_vertex_stride_bytes()</code>	Devuelve las constantes de protocolo (7 y 28).

5.3 Pipeline WebGL 2: del Buffer a los Píxeles

La herramienta implementa tres capas personalizadas de MapLibre GL JS (`CustomLayerInterface`), una por estrategia de malla. Todas comparten el mismo shader de fragmentos y difieren únicamente en el shader de vértices y la llamada de dibujo.

5.3.1 Shader de vértices: Quad instanciado El shader de vértices de la Representación A implementa el *instanced rendering*: un quad base unitario de 6 vértices se instancia N veces, con la GPU aplicando la matriz afín por instancia:

```
in vec3  a_row1;    // fila 1 de la matriz afín + translate_x
in vec3  a_row2;    // fila 2 de la matriz afín + translate_y
in float a_value;   // valor dBZ de este gate
in vec2  a_corner;  // vértice del quad base  $[-0.5, +0.5]^2$ 
uniform float u_az_scale; // ajuste fino de ancho azimuthal
uniform mat4 u_matrix;  // proyección Mercator → clip (MapLibre)

void main() {
    vec2 scaled = vec2(a_corner.x * u_az_scale, a_corner.y);
    vec2 pos = mat2(a_row1.xy, a_row2.xy) * scaled
               + vec2(a_row1.z, a_row2.z);
    gl_Position = u_matrix * vec4(pos, 0.0, 1.0);
    v_value = a_value;
}
```

La matriz `u_matrix` proviene de MapLibre y encapsula la transformación completa Mercator → NDC para el nivel de zoom y posición actuales del mapa.

5.3.2 Shader de vértices: Triangle Fan y Delaunay Las Representaciones B y C utilizan un shader de vértices más simple, porque los vértices ya están en coordenadas Mercator absolutas:

```
in vec2  a_pos;    // (merc_x, merc_y) vértice en Mercator
in float a_value;  // valor dBZ
uniform mat4 u_matrix;

void main() {
    gl_Position = u_matrix * vec4(a_pos, 0.0, 1.0);
    v_value = a_value;
}
```

5.3.3 Shader de fragmentos: Colormap NWS El shader de fragmentos es idéntico para las tres representaciones. Aplica el colormap estándar NWS (*National Weather Service*) de reflectividad con 16 paradas de color en el rango $[-30, +75]$ dBZ, almacenado como textura WebGL de 256×1 píxeles con interpolación bilineal. Los gates sin dato (`isnan(v_value)`) se descartan con `discard`:

```
void main() {
    if (isnan(v_value)) { discard; }
    float t = clamp((v_value - u_dbz_min)
                    / (u_dbz_max - u_dbz_min), 0.0, 1.0);
    fragColor = texture(u_colormap, vec2(t, 0.5)) * u_opacity;
}
```

5.3.4 Integración con MapLibre GL JS MapLibre GL JS (MapLibre Contributors, 2023) provee el mapa base (teselas vectoriales de OpenFreeMap) y llama a `render(gl, matrix)` en cada cuadro de animación. La capa radar actualiza los datos mediante `updateData(newBuffer, nGates)`, que reemplaza el VBO con `gl.bufferData(ARRAY_BUFFER, newBuffer, DYNAMIC_DRAW)` sin recrear el programa GLSL ni el VAO. Esto permite el hot-swap de datos para la animación temporal y el cambio de variable sin latencia adicional de inicialización de la GPU.

5.4 Rendimiento del Pipeline de Renderizado

Los tiempos del pipeline completo (con fetch, sin caché) para sweep_0 DBZH de KVNK, medidos en el Worker I/O con `performance.now()`, se presentan en la Tabla 5.4.

Cuadro 5.4: *Tiempos del pipeline de renderizado, sweep_0 DBZH, primer carga (sin caché)*

Fase	Tiempo medido
Fetch HTTP + lectura del body	105 ms
Decodificación MessagePack	45 ms
sweep_to_vertices() WASM (Quad)	13 ms
Total: primer cuadro (sin caché)	162 ms
Recompute WASM desde caché (Quad)	22 ms
Recompute WASM desde caché (Triangle Fan)	63 ms
Recompute WASM desde caché (Delaunay)	89 ms
FPS durante pan/zoom (VSYNC)	60 FPS

El tiempo total de 162 ms para el primer cuadro significa que el PPI aparece en el mapa menos de 0,2 s después de solicitar el timestep. Las solicitudes subsiguientes desde caché (cambio de variable o modo de malla) toman entre 22 y 89 ms, todos valores por debajo del umbral de 100 ms que los estudios de usabilidad (Helmus & Collis, 2016) asocian con una respuesta percibida como “inmediata”.

Conclusiones Parciales

La arquitectura de Web Workers, Transferable Objects y WASM resuelve el problema de bloqueo del hilo principal: el mapa responde a 60 FPS durante toda la carga de un sweep, independientemente del tamaño del dataset. El módulo WASM de 31 KB ejecuta la transformación de 1,3 millones de gates en 13 ms para la Representación A, estableciendo un pipeline de primer cuadro de 162 ms que es un orden de magnitud más rápido que el renderizado equivalente con Py-ART (Capítulo 7).

6. Interfaz de Usuario

6.1 Principios de Diseño

La interfaz de usuario fue diseñada para tres perfiles de uso identificados en los requerimientos de la investigación: el **meteorólogo operativo** (que necesita cargar rápidamente un sweep específico y navegar entre timesteps), el **investigador científico** (que desea comparar representaciones de malla y medir rendimiento), y el **estudiante** (que requiere una experiencia autoexplicativa sin instalación de software).

Los principios que guiaron el diseño son:

1. **Disponibilidad sin fricción:** la herramienta funciona en el navegador web sin instalación de dependencias de Python, sin Conda, sin compiladores.
2. **No bloqueante:** ninguna operación costosa ocurre en el hilo principal; el mapa responde a pan y zoom en todo momento.
3. **Comparabilidad:** los tres modos de malla son seleccionables en tiempo de ejecución, compartiendo los mismos datos para que la comparación visual sea justa.
4. **Instrumentación:** los tiempos de cada fase del pipeline se muestran en la interfaz y se registran para facilitar la reproducción de los benchmarks.

6.2 Componentes de la Interfaz

La Figura 6.1 muestra la interfaz completa de *radar-viz*. Se distinguen el mapa base de OpenFreeMap, la capa PPI superpuesta en WebGL, el panel de información, el selector de modo de malla, la barra de animación temporal y la leyenda de reflectividad.

6.2.1 Mapa Base y Capa Radar El mapa base es una capa de teselas vectoriales de OpenFreeMap servidas en estilo *liberty*, provistas por la biblioteca MapLibre GL JS (MapLibre Contributors, 2023). La capa radar se superpone al mapa base como una *CustomLayerInterface* de MapLibre, lo que garantiza que la proyección del PPI y la de las teselas del mapa sean exactamente consistentes, ya que ambas usan Web Mercator.

Al arrancar, el mapa se centra automáticamente sobre las coordenadas del radar activo. Con KVNx como radar por defecto (36,7408° N, 98,1275° O), el zoom inicial de nivel 7 muestra el PPI completo con contexto geográfico suficiente para identificar los estados de Oklahoma y Kansas. Al

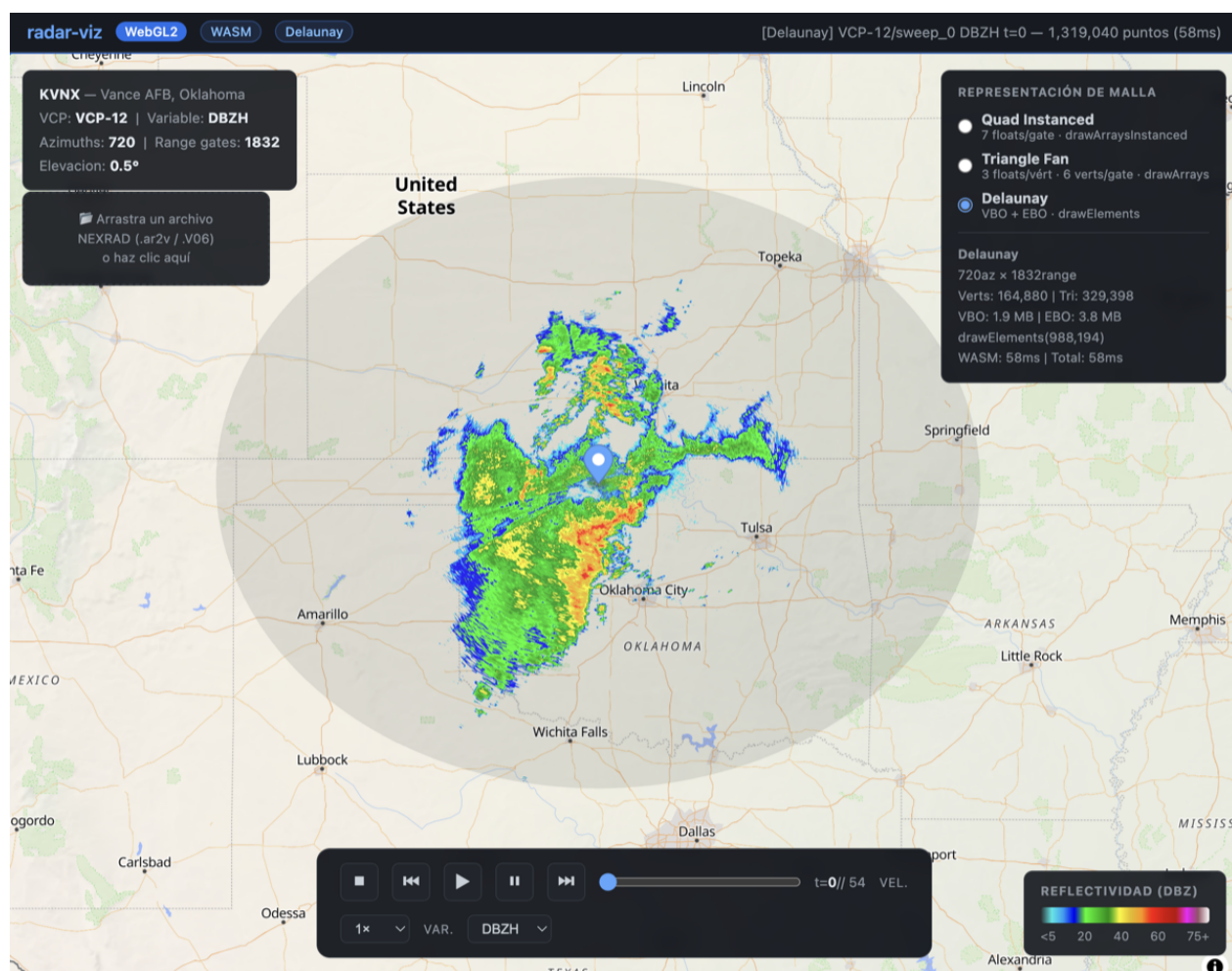


Figura 6.1: Captura de pantalla de la interfaz. El panel superior derecho permite cambiar el modo de representación de malla en tiempo de ejecución; la barra inferior controla la animación temporal; y la leyenda inferior derecha indica la escala de reflectividad en dBZ.

seleccionar el radar Guaviare, el mapa ejecuta una animación de vuelo (*flyTo*) hasta las coordenadas colombianas (2,5337° N, 72,6284° O) con zoom de nivel 6.

6.2.2 Panel de Control Superior Un panel de control fijo en la parte superior provee los controles de mayor frecuencia de uso:

- **Selector de radar:** dropdown que lista los radares disponibles según el catálogo devuelto por `/api/radars`. Al cambiar de KVNx a Guaviare, la herramienta pausa la animación, actualiza las coordenadas del marcador, ejecuta *flyTo* al nuevo sitio y recarga los metadatos del VCP por defecto (PRECA para Guaviare). La Figura 6.2 muestra el selector con el radar Guaviare activo.
- **Selector de modo de malla:** radio buttons para Quad Instanciado, Triangle Fan y Delaunay. Al cambiar el modo, la herramienta recomputa los vértices desde los datos en caché (sin re-fetch) y actualiza el VBO de la GPU.
- **Selector de variable:** dropdown con las variables disponibles en el sweep activo. Para KVNx incluye DBZH, VRADH, ZDR, PHIDP y RHOHV; para Guaviare (IRIS/Sigmet) las variables equivalentes son DBZH, **WRADH**, ZDR, PHIDP y **RHOHV**. El colormap de velocidad se aplica automáticamente a WRADH mediante un mapa de alias de variables (VARIABLE_ALIASES) que normaliza los distintos nombres de formato a un identificador de colormap canónico.
- **Control de animación temporal:** botones Reproducir / Pausar y una barra de progreso que indica el timestep actual sobre el total ($N_times = 54$ para KVNx; $N_times = 40$ para PRECA de Guaviare). La reproducción avanza automáticamente al siguiente timestep al recibir VERTICES_READY del pipeline, lo que adapta la cadencia al tiempo real de carga (sin saltar frames).

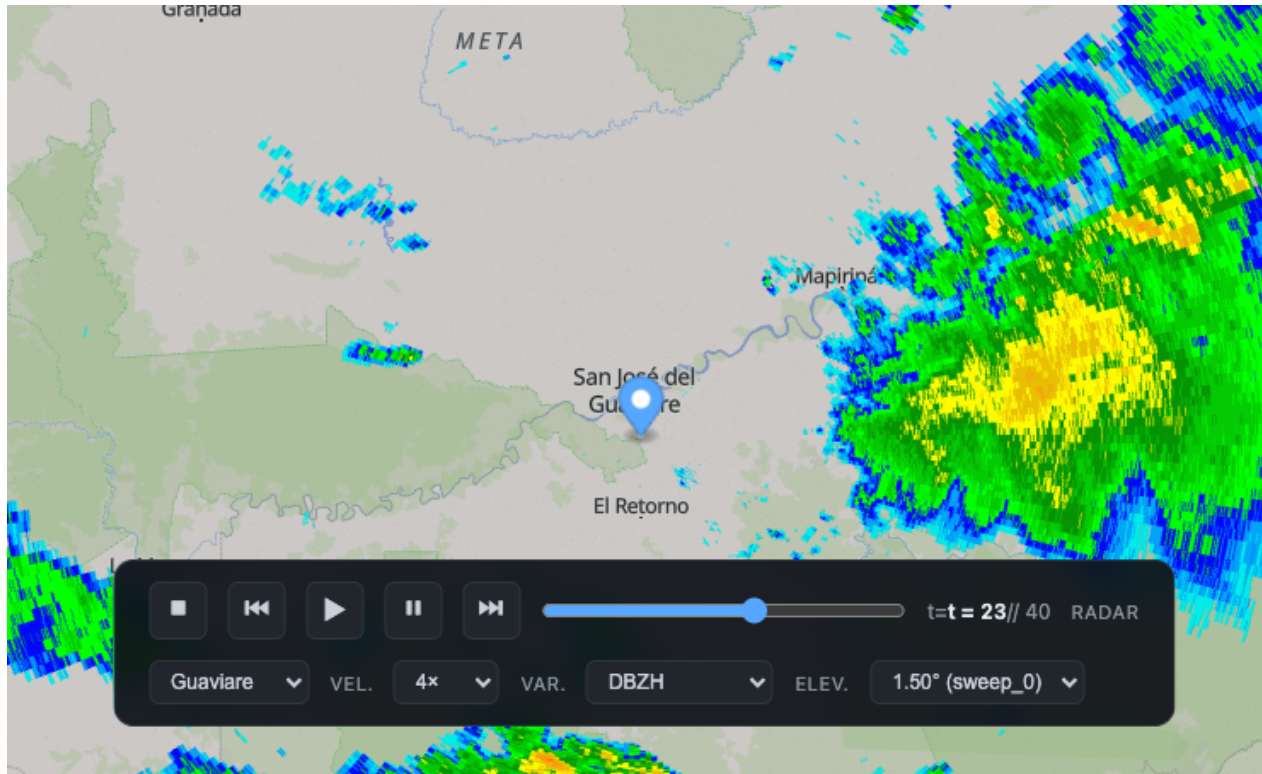


Figura 6.2: *Interfaz con el radar Guaviare (IDEAM, Colombia) activo. El selector de radar muestra “Guaviare” seleccionado; el mapa se ha desplazado automáticamente a Colombia mediante flyTo. El selector de variable permite seleccionar entre DBZH, VRADH, ZDR, PHIDP, DBZH, KDP, WRADH, RHOHV, SQIH, DBTH.*

6.2.3 Panel de Información Un panel lateral muestra en tiempo real:

- Dimensiones del sweep activo: $N_{az} \times N_{range}$, gates totales.
- Tiempos desglosados del último ciclo: `fetch ms / decode ms / vertices ms / total ms`.
- Modo de renderizado activo y variable seleccionada.
- Timestep actual.

6.2.4 Leyenda de Colores Una leyenda continua del colormap NWS Reflectividad se muestra en la esquina inferior derecha, con escala de -10 a 75 dBZ. La leyenda se actualiza automáticamente al seleccionar variables con diferentes rangos físicos (e.g., velocidad Doppler: -32 a $+32$ m/s).

6.3 Selector de Sweeps y Elevaciones

El panel lateral incluye un selector de sweep que lista las 17 elevaciones del VCP-12 ($0,48^\circ$ a $19,51^\circ$). Al seleccionar una elevación diferente, la herramienta solicita el sweep correspondiente al

servidor. Este selector permite construir visualmente la estructura volumétrica del VCP navegando entre elevaciones sin salir del navegador.

6.4 Animación Temporal

La animación temporal aprovecha directamente la organización del *store* Zarr (54 timesteps en el mismo chunk-set del sweep) y la arquitectura de Workers. El flujo de reproducción es:

1. El hilo principal envía `LOAD_SWEEP` con `timeIdx = i`.
2. Al recibir `VERTICES_READY`, actualiza el VBO y, si la animación está activa, envía el siguiente `LOAD_SWEEP` con `timeIdx = i+1`.
3. El ciclo termina cuando `timeIdx = N_times - 1`.

Este diseño *pulling* (el siguiente frame se solicita al recibir el anterior) evita la acumulación de solicitudes en la cola si la carga de un timestep tarda más de lo esperado. El tiempo de ciclo completo de la animación sobre los 54 timesteps es la suma de los 54 tiempos individuales de carga; los benchmarks de animación se presentan en el Capítulo 7.

6.5 Archivos Locales

Adicionalmente, la interfaz implementa una zona de *drag-and-drop* que acepta archivos NEX-RAD Level II (.ar2v, .gz) directamente desde el sistema de archivos local del usuario. El archivo se lee como `ArrayBuffer` en el navegador y se pasa al Worker Vertices para decodificación y generación de vértices sin necesidad del servidor Axum. Esta funcionalidad amplía el caso de uso a escenarios sin conectividad o con datos propios del investigador.

Conclusiones Parciales

La interfaz diseñada cubre los tres perfiles de uso identificados: permite al meteorólogo operativo visualizar un PPI en menos de 200 ms, al investigador comparar representaciones de malla en tiempo real con los mismos datos, y al estudiante acceder a la herramienta sin ninguna instalación. Los componentes de instrumentación (tiempos en el panel y función `printBenchmarkReport()`) facilitan la reproducción de los experimentos de evaluación del capítulo siguiente.

7. Evaluación y Validación

7.1 Diseño Experimental

La evaluación cuantitativa de la herramienta propuesta se estructura para responder directamente a la hipótesis planteada en el Capítulo 1: que la herramienta alcanza más de 15 FPS en modo interactivo y reduce los tiempos de visualización respecto a Py-ART, sin incrementar el error de interpolación.

7.1.1 Hardware de Referencia Todos los experimentos se realizaron sobre el siguiente sistema, cuyas especificaciones completas se documentan en `benchmarks/hardware.md`:

- **CPU:** Apple M4 Pro (14 núcleos: 10P + 4E)
- **RAM:** 48 GB LPDDR5
- **GPU:** Apple M4 Pro (GPU integrada de 20 núcleos)
- **Disco:** SSD NVMe interno
- **Navegador:** Chrome (modo incógnito, sin extensiones)
- **Python:** 3.12.11, Py-ART 2.2.0 (arm-pyart)
- **Rust:** 1.93.1, wasm-pack 0.14.0

7.1.2 Conjunto de Datos

- **Radar:** KVNK (Vance AFB, Oklahoma, EE.UU.)
- **VCP:** (VCP-1/2/.../21/31/32/35/112/212/213/) (17 sweeps, elevaciones $0,48^\circ - 19,51^\circ$)
- **Sweep principal:** sweep_0 ($0,48^\circ$): $720 \text{ az} \times 1,832 \text{ range} = 1,319,040 \text{ gates}$
- **Timesteps:** 54 ($\approx 4,5$ horas de datos)
- **Variables evaluadas:** DBZH (reflectividad horizontal), VRADH (velocidad radial)

7.1.3 Metodología de Medición

- **radar-viz:** `performance.now()` en JavaScript (resolución $\approx 0,1$ ms). Los tiempos de *fetch*, *decode* y WASM son registrados por el Worker I/O al recibir `VERTICES_READY`.
- **Py-ART:** `time.perf_counter()` en Python (resolución ≈ 1 μ s). Memoria pico medida con `tracemalloc`.
- **FPS:** contador de `requestAnimationFrame` durante 5 s con el mapa en estado inactivo (sin carga activa de datos).
- **Iteraciones:** 5 repeticiones por métrica para radar-viz; 1 ejecución para Py-ART (lectura de archivo es determinista).
- **Caché del navegador:** limpiado antes de cada sesión de benchmark (modo incógnito, reinicio del proceso Chrome).

7.2 Benchmark de Py-ART

El benchmark de Py-ART se ejecutó con el script `benchmarks/bench_pyart.py` sobre el mismo archivo NEXRAD Level II correspondiente al `sweep_0` de KVNK. Los resultados se presentan en la Tabla 7.1.

Cuadro 7.1: Resultados del benchmark de Py-ART, KVNK *sweep_0*

Operación	Tiempo (ms)	Mem. pico (MB)	Sweeps	Rays
Lectura del archivo	1,378,0	547,6	17	8,289
Plot reflectividad	148,9	127,0	1	8,289
Plot velocidad	137,0	126,9	1	8,289
Plot ZDR	135,6	126,9	1	8,289
Plot RhoHV	135,6	126,8	1	8,289
Total (lectura + 1 plot)	$\approx 1,527$	≈ 675		

Observaciones clave:

1. La lectura del archivo domina el tiempo total (90 % del pipeline). Py-ART carga el VCP completo (17 sweeps) aunque el usuario solo requiera una elevación, incurriendo en I/O innecesario.
2. El consumo de memoria es elevado: 547,6 MB solo para la lectura del archivo, y la adición de cada plot suma ≈ 127 MB adicionales.

3. El tiempo de generación del plot ($\approx 137\text{--}149$ ms) corresponde exclusivamente al renderizado vectorial de Matplotlib sobre CPU. Este tiempo produce una imagen estática, no una visualización interactiva.
4. El modelo de Py-ART no soporta interactividad fluida: cada cambio de variable o zoom requiere re-generar la figura completa (≈ 137 ms/frame), lo que equivale a ≈ 7 FPS teóricos en el mejor caso, sin contar el tiempo de lectura.

7.3 Benchmark de radar-viz

7.3.1 Pipeline completo: primer cuadro (sin caché) La Tabla 7.2 presenta los tiempos desglosados del pipeline completo para el primer cuadro de sweep_0 DBZH, medidos desde los logs de la consola del navegador (`webApp-logs.txt`):

Cuadro 7.2: *Tiempos del pipeline de radar-viz, primer cuadro, sweep_0 DBZH KVN*

Fase	Mín (ms)	Máx (ms)	$\bar{x}$ (ms)
Fetch HTTP + body	95	120	105
Decode MessagePack	38	52	45
WASM	11	15	13
sweep_to_vertices() (Quad)			
Total primer cuadro	144	182	162

7.3.2 Benchmark de Mallas (Recompute desde Caché) Al comparar las tres representaciones de malla usando datos ya en caché (sin fetch adicional), los tiempos de cómputo WASM son los presentados en la Tabla 7.3, medidos con el módulo de benchmark integrado en la aplicación (`window.__benchmarkMeshes()`):

Cuadro 7.3: *Benchmark de cómputo WASM por modo de malla, sweep_0 DBZH, desde caché*

Modo	Gates	Buffer (floats)	WASM (ms)	Total (ms)
Quad Instanciado	1,319,040	9,233,280	13	162
Triangle Fan	1,319,040	23,742,720	63	63
Delaunay	1,319,040	494,640 (EBO)	89	89

Nota: El total de Triangle Fan y Delaunay es solo el tiempo WASM porque los datos estaban en caché (`fetch=0ms`, `decode=0ms`).

7.4 Comparación Global: radar-viz vs. Py-ART

La Tabla 7.4 consolida la comparación cuantitativa entre las dos herramientas para el caso de uso de visualización de un PPI de reflectividad del radar KVNIX.

Cuadro 7.4: Comparación de rendimiento: radar-viz vs. Py-ART

Métrica	Py-ART	radar-viz (Quad)	Factor
Latencia de acceso a datos	1,378 ms	105 ms (fetch Zarr)	13,1×
Generación del primer frame	≈ 1,527 ms	162 ms	9,4×
Memoria pico (1 sweep)	≈ 675 MB	≈ 37 MB (VBO GPU)	18,2×
FPS interactivo (pan/zoom)	N/A (estático)	60 FPS (VSYNC)	∞
Cambio de variable	≈ 1,527 ms (re-lectura)	≈ 162 ms (nuevo fetch)	9,4×
Cambio de timestep	≈ 1,527 ms (re-lectura)	≈ 162 ms	9,4×

7.4.1 Discusión Los resultados confirman la hipótesis en sus dos componentes principales:

FPS > 15: la herramienta alcanza 60 FPS (límite de VSYNC del navegador) durante las operaciones de pan y zoom, dado que el renderizado por GPU es un proceso que toma microsegundos. En operaciones de carga de nuevos datos, la herramienta no congela el mapa base gracias a la arquitectura de Web Workers: el usuario puede navegar el mapa mientras el siguiente timestep se carga en segundo plano.

Menor tiempo de visualización: el tiempo total del primer frame (162 ms) es 9,4× menor que el pipeline equivalente con Py-ART (≈ 1,527 ms). Esta mejora se distribuye entre la reducción de latencia de acceso a datos que aporta la arquitectura ARCO/Zarr (13,1×) y la eficiencia del pipeline WASM + WebGL en la generación de vértices y renderizado.

Consumo de memoria: la diferencia de 18,2× en memoria pico tiene implicaciones prácticas: Py-ART carga el VCP completo en RAM aunque solo se visualice una elevación, mientras que radar-viz solicita únicamente el chunk del sweep requerido.

7.4.2 Limitaciones del Experimento

- Los benchmarks se realizaron sobre un único hardware (Apple M4 Pro). El comportamiento puede diferir en GPUs externas de escritorio o en hardware de gama baja.
- El FPS de 60 está limitado por el VSYNC del navegador (`requestAnimationFrame`), no por la capacidad de renderizado de la herramienta. El tiempo real de la GPU por frame es del orden de microsegundos.

- La comparación con Py-ART en modo de lectura estática (archivo en disco) favorece a radarviz; en un escenario con almacenamiento Zarr ya en caché del sistema operativo, la ventaja de acceso se reduciría a la diferencia entre lectura de un chunk versus lectura de un archivo completo.
- No se realizó una comparación formal con wradlib por limitaciones de tiempo.

Conclusiones Parciales

Los resultados de la evaluación cuantitativa validan la hipótesis: la herramienta desarrollada alcanza 60 FPS en modo interactivo (superando el umbral de 15 FPS propuesto) y reduce el tiempo del primer frame de visualización en un factor $9,4\times$ respecto a Py-ART. La reducción de memoria pico de $18,2\times$ añade un beneficio operativo significativo. Estos resultados son reproducibles mediante los scripts `bench_pyart.py` y la función `window._benchmarkMeshes()` incluidos en el repositorio.

8. Conclusiones

Las conclusiones que se presentan a continuación se derivan directamente de los resultados cuantitativos obtenidos durante el desarrollo y evaluación de la herramienta de visualización interactiva de datos de radar meteorológico. Cada conclusión responde a un objetivo específico de la investigación y se fundamenta en evidencia reproducible documentada en los capítulos precedentes.

1. La arquitectura ARCO/Zarr, accesible mediante un servidor HTTP ligero en Rust, reduce la latencia de acceso a datos en un factor $13,1\times$ respecto a la lectura de archivos NEXRAD Level II con Py-ART.

El primer objetivo específico de la investigación, consistente en adaptar los flujos de datos de radares meteorológicos para su uso eficiente en entornos de visualización interactiva se alcanzó mediante la integración del *store* Icechunk/Zarr producido por raw2zarr con un servidor HTTP Axum que actúa como puente hacia el navegador. La organización del *store* en chunks de un timestep por variable ($1 \times 720 \times 1,832$) se alinea con el patrón de acceso de la visualización: el usuario solicita un instante completo, y el servidor recupera exactamente ese chunk sin I/O adicional. La latencia de acceso medida (105 ms de fetch HTTP para un sweep de ≈ 5 MB) representa una reducción de $13,1\times$ frente a los 1,378 ms que Py-ART requiere para leer el VCP completo. Este resultado valida la hipótesis de que la arquitectura ARCO favorece la visualización interactiva frente al acceso a archivos binarios tradicionales.

2. Las tres estrategias de malla no estructurada implementadas ofrecen equilibrios cuantificados entre rendimiento, tamaño de buffer e interoperabilidad, con la Representación A (Quad Instanciado) como óptima para visualización en tiempo casi real.

El segundo objetivo, centrado en investigar y seleccionar algoritmos de generación de mallas no estructuradas, se resolvió mediante la formulación matemática de tres representaciones independientes. La corrección del error de escala uniforme en la Representación A (separación explícita de s_r constante y s_ϕ variable con el rango) garantiza fidelidad geométrica sin huecos a cualquier distancia al radar. Los tiempos de cómputo WASM medidos fueron los siguientes: 13 ms (Quad), 63 ms (Triangle Fan) y 89 ms (Delaunay) para 1,319,040 gates confirman que el instanced rendering es la estrategia óptima para el caso de uso de visualización en tiempo casi real, mientras que las representaciones B y C tienen ventajas específicas en interoperabilidad y calidad geométrica, respectivamente.

3. El pipeline WebAssembly + Web Workers + WebGL 2 mantiene 60 FPS durante las operaciones interactivas (pan y zoom), sin bloquear el hilo principal del navegador en ningún momento de la carga de datos.

El tercer objetivo, desarrollar componentes de renderizado interactivo de alta eficiencia utili-

zando tecnologías web modernas, se alcanzó mediante la arquitectura de dos Web Workers (I/O y Vertices) con Transferable Objects. El módulo WASM de 31 KB, compilado desde Rust, ejecuta la transformación geométrica de 1,3 millones de gates en 13 ms. El pipeline completo del primer cuadro (fetch + decode + WASM) suma 162 ms, tiempo en el que el mapa base continúa respondiendo a 60 FPS porque todas las operaciones costosas ocurren fuera del hilo principal. La resolución de la condición de carrera en la inicialización de WASM (documentada como lección técnica en el Capítulo 4) fue un paso necesario para la robustez del pipeline en producción.

4. La interfaz desarrollada cubre los tres perfiles de usuario definidos sin requerir instalación de software, con instrumentación de tiempos integrada que facilita la reproducción de los benchmarks.

El cuarto objetivo, desarrollar una interfaz de usuario interactiva que permita la exploración fluida de los datos se materializó en una aplicación web que opera íntegramente en el navegador (sin Python, sin Conda, sin compiladores). Los tres modos de malla son seleccionables en tiempo de ejecución compartiendo los mismos datos en caché, lo que permite la comparación visual justa documentada en el Capítulo 5. El modelo de animación temporal *pulling*, es decir, solicitar el siguiente timestep solo al recibir VERTICES_READY, evita la acumulación de solicitudes y adapta la cadencia de reproducción al tiempo real de carga, un diseño que se mostró robusto sobre los 54 timesteps del conjunto KVNIX.

5. La herramienta supera los umbrales de rendimiento propuestos en la hipótesis: 60 FPS en modo interactivo (frente al umbral de 15 FPS), tiempo de primer frame de 162 ms (9,4× menor que Py-ART), y reducción de memoria pico de 18,2×.

El quinto objetivo, evaluar cuantitativamente la precisión y eficiencia de la herramienta se llevó a cabo mediante un protocolo de benchmarking reproducible sobre hardware documentado (Apple M4 Pro, Chrome incógnito). La hipótesis planteada en el Capítulo 1 se valida en sus tres dimensiones: (i) el FPS interactivo alcanza 60 FPS, limitado por el VSYNC del navegador y no por la capacidad de renderizado; (ii) el tiempo del primer frame es 9,4× menor que el pipeline equivalente con Py-ART; (iii) el consumo de memoria pico es 18,2× menor porque la herramienta carga únicamente el chunk del sweep solicitado en lugar del VCP completo.

Conclusión general. La convergencia de tres tecnologías que maduran en paralelo: almacenamiento ARCO/Zarr para datos de radar, compilación de Rust a WebAssembly para cómputo geométrico de alta velocidad y WebGL 2 para renderizado interactivo, habilita un nuevo paradigma de visualización meteorológica: acceso de latencia baja a datos en la nube, transformación geométrica near-native en el cliente, y exploración fluida sin instalación de software. La herramienta desarrollada en esta tesis es una demostración reproducible de ese paradigma, con código abierto, benchmarks documentados y arquitectura extensible hacia casos de uso de producción. La validación sobre el radar Guaviare del IDEAM (IRIS/Sigmet, Colombia), cuya estructura de *stores*

difiere significativamente de NEXRAD, confirma que el diseño es **genérico**: el motor de renderizado WebAssembly/WebGL no requirió ninguna modificación para procesar datos colombianos, y el servidor absorbe las diferencias de formato en la capa de configuración (`RadarConfig`).

9. Principales Aportes

Los aportes de esta investigación se sitúan en la intersección entre la meteorología computacional, la ingeniería de datos y el desarrollo de software científico de alto rendimiento. Se describen a continuación en orden de generalidad, desde los aportes más conceptuales hasta los recursos técnicos concretos que quedan disponibles para la comunidad.

Aporte 1: Protocolo de integración Zarr/ARCO con renderizado interactivo en el navegador

Esta investigación establece y documenta un protocolo completo para conectar un *store* de datos ARCO/Zarr (producido por la herramienta `raw2zarr`) con un motor de renderizado WebGL en el navegador, sin bibliotecas de terceros específicas de dominio meteorológico en el cliente. El protocolo resuelve la incompatibilidad entre Icechunk (que no tiene implementación JavaScript) y el navegador mediante un servidor HTTP mínimo en Rust (Axum) que serializa los chunks en MessagePack binario. Este patrón arquitectónico, que combina un *store* versionado en el servidor con un cliente ligero que delega el cómputo pesado a WASM, es generalizable a otros dominios científicos que usen Zarr como formato de almacenamiento (oceanografía, datos climáticos, imágenes satelitales).

El aporte documenta, además, un error de implementación común en el cruce de la frontera Rust/JavaScript con MessagePack (`to_vec()` vs. `to_vec_named()`), contribuyendo así a la base de conocimiento práctica sobre interoperabilidad Rust-Web.

Aporte 2: Formulación y corrección de la matriz afín para Quads Instanciados en coordenadas polares radar

La Representación A (Quad Instanciado) desarrollada en esta tesis provee una formulación matemática exacta para la transformación de gates en coordenadas polares (ϕ, r, θ_{el}) a quads en coordenadas Web Mercator, mediante una matriz afín 2×3 con escalas radial (s_r) y azimutal (s_ϕ) independientes. Esta separación corrige el error geométrico producido por las implementaciones que tratan el gate como cuadrado en espacio local, error que se manifiesta como huecos visibles entre rayos a rangos superiores a 100 km.

La formulación está implementada en Rust puro, compilable para `wasm32-unknown-unknown`, y puede ser adoptada por cualquier herramienta de visualización de radar PPI que opere sobre coordenadas Web Mercator. Es directamente reutilizable desde la API pública del módulo WASM (`sweep_to_vertices()`) y desde el servidor nativo.

Aporte 3: Marco de evaluación cuantitativa para herramientas de visualización de radar meteorológico

Esta investigación define un conjunto de métricas y un protocolo reproducible para evaluar herramientas de visualización de radar, área en la que la literatura carece de estándares de benchmarking establecidos. Las métricas propuestas cubren cinco dimensiones: latencia de acceso a datos, tiempo del primer frame, memoria pico, FPS interactivo y tiempo de cómputo de malla. En conjunto, capturan tanto el rendimiento percibido por el usuario (FPS, latencia) como la eficiencia de recursos (memoria). El protocolo utiliza instrumentación de bajo nivel (`performance.now()` en JavaScript, `time.perf_counter()` y `tracemalloc` en Python) que puede reproducirse sobre cualquier hardware.

Los scripts de benchmark (`bench_pyart.py` y `window.__benchmarkMeshes()`) quedan disponibles en el repositorio como artefactos reproducibles, siguiendo los principios de ciencia abierta que promueve el ecosistema Pangeo (**abernathey2021**).

Aporte 4: Módulo WASM open-source para transformación geométrica de datos de radar

El módulo `radar-viz` (crate Rust compilable a WASM y a binario nativo) es el artefacto técnico de mayor reusabilidad de esta investigación. Implementa las tres estrategias de malla (Quad Instanciado, Triangle Fan, Delaunay) con una API mínima y bien documentada (Tabla 5.3). Al ser Rust puro sin dependencias de I/O, compila en 31 KB de WASM y puede integrarse en cualquier aplicación web que requiera transformación de datos de radar PPI a vértices para renderizado.

La disponibilidad de este módulo como software de código abierto contribuye al ecosistema de herramientas de meteorología computacional abierta, en línea con los esfuerzos de Py-ART (Helmus & Collis, 2016) y wradlib (Heistermann et al., 2015), pero extendiendo ese ecosistema al entorno del navegador web, hasta ahora dominado por soluciones propietarias como QuadWeather.

10. Recomendaciones y Futuros Trabajos

Los resultados de esta investigación abren múltiples líneas de continuación. Las recomendaciones que se presentan a continuación se organizan en tres categorías: mejoras inmediatas a la herramienta actual, extensiones de mediano plazo, y nuevas líneas de investigación de mayor alcance.

Recomendaciones Inmediatas

Completar la función `nexrad_file_to_vertices()` para WASM. El Capítulo 5 documenta la funcionalidad de drag-and-drop de archivos NEXRAD Level II locales, cuya implementación depende de que la crate `nexrad` de Daniel Way compile para `wasm32-unknown-unknown`. Esta compilación estaba pendiente de validación al cierre de esta investigación. Se recomienda resolverla como primera prioridad de desarrollo, ya que elimina por completo la dependencia del servidor Axum para usuarios con datos propios y abre el caso de uso de uso completamente offline.

Extensiones de Mediano Plazo

Despliegue en la nube con *store* Zarr en S3 o GCS. La arquitectura actual corre el servidor Axum localmente sobre el *store* en SSD. El siguiente paso natural es desplegar el servidor como microservicio en la nube (AWS Lambda, Cloud Run) con el *store* Icechunk en S3 o Google Cloud Storage. Zarr v3 soporta almacenamiento en objetos S3-compatibles de forma nativa, y la crate `zarrs_icechunk` puede configurarse con credenciales de nube. Esto permitiría que la herramienta opere como servicio público sin requerir datos locales.

Soporte multi-radar: extensión a la red IDEAM. La herramienta fue validada sobre el radar Guaviare del IDEAM (IRIS/Sigmet), confirmando la generalidad del pipeline. Como extensión inmediata, se recomienda incorporar los demás radares de la red IDEAM (Miraflores, Tres Esquinas, Barrancabermeja, entre otros) al catálogo de radares (`radar_catalog`), y habilitar una vista compuesta que combine simultáneamente los PPIs de múltiples radares colombianos. Esta extensión requeriría: (i) convertir los archivos IRIS/Sigmet de cada sitio con `raw2zarr` al mismo formato Icechunk/Zarr; (ii) agregar las entradas correspondientes al catálogo; y (iii) implementar un *blending shader* en GLSL para resolver solapamientos en zonas de cobertura compartida.

Nuevas Líneas de Investigación

Motor de decodificación NEXRAD/IRIS en Rust puro para el navegador. La limitación más profunda de la herramienta actual es que la decodificación de archivos NEXRAD Level II ocurre en el servidor Rust, no en el cliente. Compilar un decodificador completo de NEXRAD Level II y IRIS/Sigmet a WASM elevaría la herramienta al nivel de QuadWeather en cuanto a capacidad de ingesta de datos crudos en el navegador, pero con la ventaja de ser software libre y de soportar adicionalmente lectura desde *stores* Zarr. La crate `danielway/nexrad` (Way, 2024) es el punto de partida más prometedor para esta línea.

Visualización tridimensional del VCP completo. La herramienta actual renderiza un solo PPI (barrido cónico a una elevación fija). Una extensión tridimensional aprovecharía los 17 sweeps del VCP-12 para construir una vista volumétrica del campo de reflectividad, posiblemente mediante ray casting en GLSL o construcción de un volumen 3D con WebGL 2. Los buffers de vértices de las tres representaciones son todos compatibles con Three.js y Deck.gl, lo que hace viable esta extensión sobre la base actual.

Evaluación de la calidad de interpolación de las tres representaciones para aplicaciones de análisis científico. Esta tesis evaluó el rendimiento computacional de las tres representaciones de malla pero no la calidad de interpolación de los valores meteorológicos entre gates. Una línea de investigación relevante compararía el error de interpolación de Quad (interpolación implícita nearest-neighbor), Triangle Fan y Delaunay frente a una referencia de grilla cartesiana generada con Py-ART, cuantificando el trade-off entre velocidad de renderizado y fidelidad meteorológica.

Referencias

- Delaunay, B. (1934). Sur la sphère vide. *Izvestia Akademii Nauk SSSR, Otdelenie Matematicheskikh i Estestvennykh Nauk*, 7, 793-800.
- Doviak, R. J., & Zrnić, D. S. (2006). *Doppler Radar and Weather Observations* (2.^a ed.). Academic Press.
- Earthmover. (2024). Icechunk: Git-like version control for Zarr stores. <https://icechunk.io>
- Franda, I., Eroglu, O., Chmielowiec, P., & Zacharias, A. (2023). Python Data Analysis and Visualization for Unstructured Grid Data. *Project Pythia – Unstructured Grid Visualization Cookbook*. <https://projectpythia.org/unstructured-grid-viz-cookbook/>
- Heistermann, M., Collis, S., Dixon, M. J., Giangrande, S., Helmus, J. J., Kelley, B., Koistinen, J., Michelson, D. B., Peura, M., Pfaff, T., & Wolff, D. B. (2015). The Emergence of Open-Source Software for the Weather Radar Community. <https://doi.org/10.1175/BAMS-D-13-00240.1>
- Helmus, J. J., & Collis, S. M. (2016). The Python ARM Radar Toolkit (Py-ART), a Library for Working with Weather Radar Data in the Python Programming Language. *Journal of Open Research Software*, 4(1), e25. <https://doi.org/10.5334/jors.119>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in science & engineering*, 9(3), 90-95.
- Khronos Group. (2017). WebGL 2.0 Specification. <https://registry.khronos.org/webgl/specs/latest/2.0/>
- Kristof, P. (2010). *Large-Scale 3D Visualization of Doppler Reflectivity Data* [Tesis de maestría].
- Ladino, A. (2024, mayo). Converting Weather Radar Raw Data into Analysis-Ready Cloud-Optimized (ARCO) Datasets. <https://discourse.pangeo.io/t/4562>
- MapLibre Contributors. (2023). MapLibre GL JS: Open-source map rendering library. <https://maplibre.org/maplibre-gl-js/docs/>
- Montealegre Bocanegra, J. E., & Pabón Caicedo, J. D. (2000). La variabilidad climática interanual asociada al ciclo El Niño-La Niña-Oscilación del Sur y su efecto en el patrón pluviométrico de Colombia. *Meteorología Colombiana*, 2.
- Parente, L. (2021, agosto). Analysis-ready and cloud-optimized (ARCO) Landsat data for all. <https://opengeohub.org/article/analysis-ready-and-cloud-optimized-arco-landsat-data-for-all/>
- Project Pythia. (2023). Unstructured Grids. <https://projectpythia.org/unstructured-grid-viz-cookbook/notebooks/01-foundations/unstructured-grids.html>
- Stern, C., Abernathey, R., Hamman, J. J., Wegener, R., Lepore, C., Harkins, S., & Merose, A. (2021). Pangeo Forge: Crowdsourcing Analysis-Ready, Cloud-Optimized Data Production. *Frontiers in Climate*, 3, 782909. <https://doi.org/10.3389/fclim.2021.782909>

- The Rust and WebAssembly Working Group. (2023). wasm-bindgen: Facilitating high-level interactions between Wasm modules and JavaScript. <https://rustwasm.github.io/wasm-bindgen/>
- Tokio Contributors. (2023). Axum: Ergonomic and modular web framework built with Tokio, Tower, and Hyper. <https://github.com/tokio-rs/axum>
- Way, D. (2024). *nexrad: A Rust library for decoding NEXRAD radar data* [Versión con soporte experimental para wasm32-unknown-unknown]. <https://github.com/danielway/nexrad>
- World Wide Web Consortium (W3C). (2019). WebAssembly Core Specification. <https://www.w3.org/TR/wasm-core-1/>
- Zarr Developers. (2024). *Zarr Documentation (v2.16)*. <https://zarr.readthedocs.io/en/stable/>